System Requirements

SeatSphere: Movie Ticket Booking System

Danielle Kemp

Department of Informatics, Indiana University

INFO-C 450: System Design

May 1, 2026

Table of Contents

Customer Problem Statements and System Requirements 4
Problem Statement 4
Glossary of Terms 4
System Requirements 6
Functional Requirements 6
Non-Functional Requirements 7
User Interface Requirements 8
Functional Requirements Specification 9
Stakeholders 9
Use Cases 9
Use Case Diagram 11
Class Diagram 11
Data Types and Operation Signatures 11
Enumerations and Custom Data Types 15
Composition 16
Class Diagram 17
System Sequence Diagram and Activity Diagram 18
System Sequence Diagrams 18
SSD1: 18
SSD2: 20
Activity Diagrams 22
Activity Diagram 1 22
Activity Diagram 2 24
User Interface Specification 26
Preliminary Design 26
Use Case 1: Customer selects seats and confirms a booking 26
Use Case 2: Staff scans and validates ticket 30
Use Case 3: Theater Manager monitors Seat Availability 33
Use Case 4: Admin adds a new movie listing 37
Use Case 5: Admin Views Sales Report 41

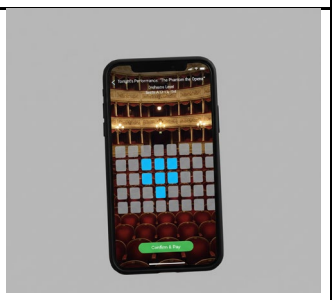
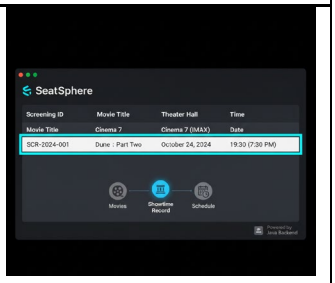
User Effort Estimation 44
Traceability Matrix 45
System Requirements 45
Use Cases 46
Traceability Matrix 46
System Architecture and System Design 47
Architectural Styles 47
Identifying Subsystems 47
Persistent Data Storage 48
Network Protocol 49
Global Control Flow 49
Execution Orders 49
Time Dependency 49
Concurrency 50
Hardware Requirements 50
Plan of Work / Development Plan 50
References 53

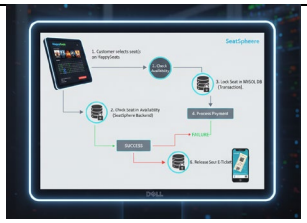
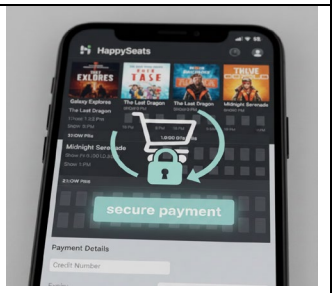
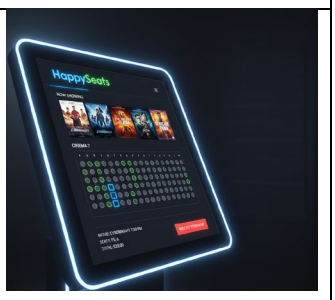
Customer Problem Statements and System Requirements

Problem Statement

A movie theater is a venue that is geared towards public film viewing and is often located in busy areas such as shopping centers. In a theater, there are a fixed number of theaters, and each theater has a specific number of seats. Currently, one can monitor the viewing of films using physical displays or sites, but this creates confusion and hence long lines of people waiting to purchase their tickets. In a theater, tickets are issued at a counter, but since there is no real-time viewing, one is likely to realize that their preferred seats are already occupied. After choosing a film, a physical tracking system is adopted to ensure that there are no double bookings. The final stage is at the ticket booth, where one pays for their chosen film using cash or credit cards, but since there is no digital system, one is likely to incur an inefficient scheduling system.

Glossary of Terms

Term	**Definition**	**Example**
SeatMap	A visual, interactive grid layout of all seats within a specific theater auditorium, color-coded to indicate real-time availability	
Showtime	A unique database record representing a scheduled movie screening tied to a specific theater hall, date, and time.	

User Profile	A profile created by a customer or theater manager to log in, manage bookings, and access features.	
Booking	The completed process of a customer selecting seats, providing information, and successfully finalizing the transaction.	
Reservation	A temporary status where selected seats are "locked" in the MySQL database to prevent double-bookings while the user completes payment.	
Payment Gateway	The integrated online service layer within the Java backend that securely processes credit or debit transactions for ticket purchases.	
HappySeats	The dedicated customer-facing Node.js frontend interface used by moviegoers for browsing and seat selection.	

SeatSphere	The administrative Java-based backend suite used by theater managers to oversee scheduling, inventory, and sales reports	

System Requirements

Functional Requirements

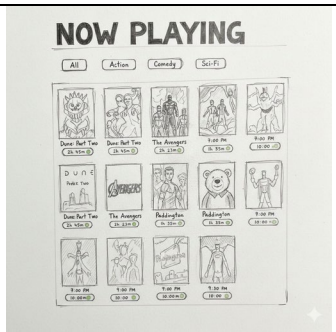
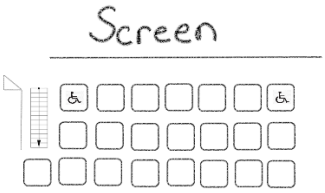
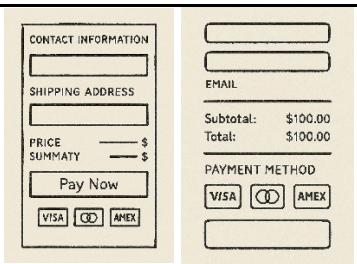
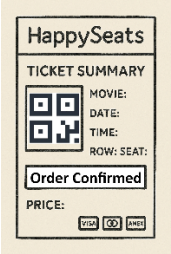
No.	Priority Weight	Description
REQ 1	High	The system must allow users to browse movies and showtimes.
REQ 2	High	The system must allow customers to be able to view a visual seat map of the theater.
REQ 3	High	The system must allow users to select and reserve specific seats.
REQ 4	High	The system shall integrate with a third-party payment gateway to process credit card transactions and return a success/failure status.
REQ 5	Medium	The system must allow users to receive a digital ticket confirmation via email or app.
REQ 6	Medium	The system shall implement a database lock on a selected seat for 10 minutes once added to a user's cart to

		prevent concurrent booking conflicts
REQ 7	Low	The system may offer suggested seats based on user preference (e.g., center or aisle).
REQ 8	Medium	The system must allow users to cancel bookings within a specified time window before showtime.

Non-Functional Requirements

No.	Priority	Category	Description
NFR 1	High	Functionality	The system must accurately track seat availability in real-time and prevent double booking of seats.
NFR 2	High	Reliability	The system must be available at least 99% of the time during peak hours to ensure customers can book tickets without errors.
NFR 3	High	Supportability	The system should be easy to maintain and update, allowing the development team to quickly fix bugs or add new movies/showtimes.
NFR 4	Medium	Usability	The system interface must be intuitive and accessible for users of all ages, including clear navigation and responsive design.
NFR 5	Medium	Performance	The system should allow pages to load in under 3 seconds, even when multiple users are browsing and booking simultaneously.

User Interface Requirements

No.	Priority	Name	Description	Sketch/Graphic
UI 1	High	Movie Browsing Dashboard	Home page displays movie titles and showtimes in a clear, card-style layout with “Book Now” buttons.	
UI 2	High	Interactive Seating Map	Seat selection page shows interactive seat map with color-coded availability: green = available, gray = occupied.	
UI 3	Medium	Payment Page	Payment page shows clear breakdown of total cost and accepted payment methods.	
UI 4	Medium	Confirmation Page	Confirmation page provides ticket details and QR code for theater entry.	
UI 5	Medium	Staff Management Suite	Staff management dashboard for theater managers to add movies and manage showtime schedules.	

Functional Requirements Specification

Stakeholders

- Customers: User who browse movies, select seats, and purchase the movie tickets.
- Theater Managers: Manages showtimes, monitors ticket availability and assist customers.
- Theater Staff: Assist customers, validate tickets, and manage theater operations.
- System Administrator: Manages the movies, pricing, users, and generates reports.
- Payment Provider: A third part service that processes payments (e.g. PayPal, Venmo, etc.).

Actors and Goals

1) Primary Actor:
    - Customer: This actor can use the system to browse movies, select seats, buy tickets, and view confirmation of ticket purchase.
    - Theater Manager: This actor can use the system to manage showtimes, monitor availability, and help customers.
2) Secondary Actor:
    - System Administrator: This actor can use the system to add and remove movies, manage prices, manage users, and view reports.
    - System: This can assign seats, process payments, generate tickets, and send confirmations.
3) Background actor: The SeatSphere database system alongside the HappySeats user application.

Use Cases

- Admin (Total: 18)
    - Add./remove movie (2)
    - Set Ticket Prices (3)

    - Manage Theaters/Halls (2)
    - Manage Users (2)
    - Login and Logout (3)
    - View Sales Reports (3)
- Customer (Total: 14)
    - Browse Movies (2)
    - View Showtimes (2)
    - Select Seats (3)
    - Book Tickets (3)
    - Make Payment (2)
    - View and Download Ticket (2)
- Theater Staff (Total: 10)
    - View Seat Status (2)
    - Check Bookings (2)
    - Assist Customer Booking (2)
    - Validate Tickets (2)
    - Login and Logout (2)
- Theater Manager (Total: 6)
    - Manage Showtimes (2)
    - Monitor Seat Availability (2)
    - Assist Customers / Staff (2)
- System (Total: 8)
    - Assign Available Seats (2)

- Process Payment (2)
- Generate QR Code Ticket (2)
- Send Confirmation Email (2)

Use Case Diagram

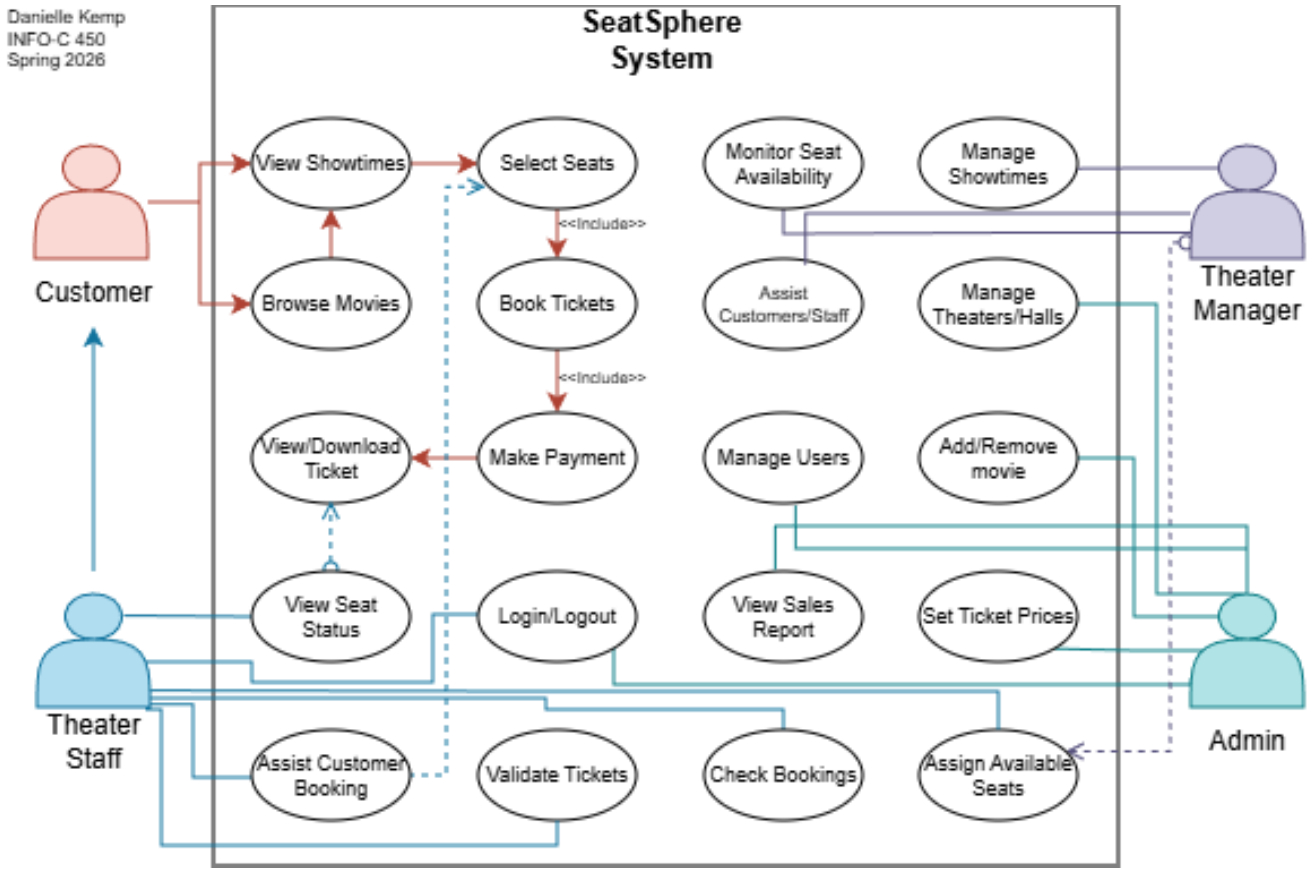


Figure 1: SeatSphere Functional Use Case Diagram.
Designed by Danielle Kemp via draw.io

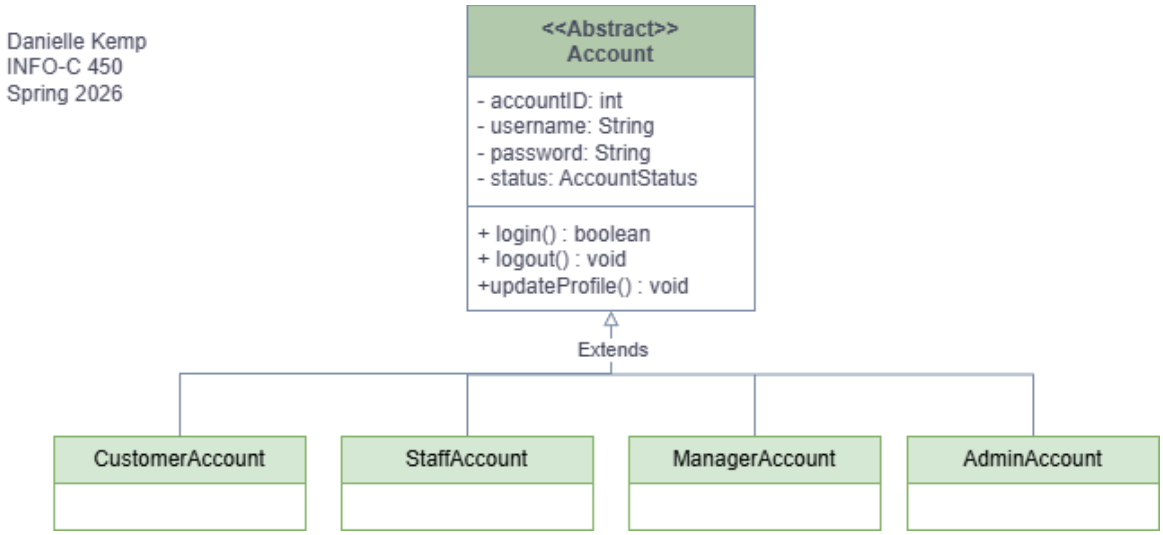
Class Diagram

Data Types and Operation Signatures

<u>Account</u>

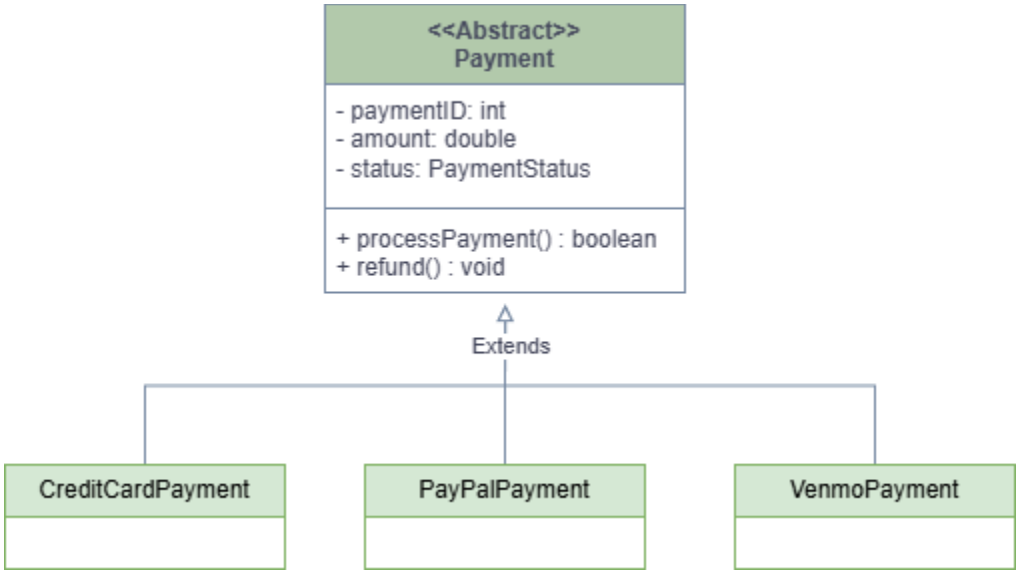
To avoid code duplication and improve scalability, we define an abstract Account class as the base for all users (Customers, Admins, Staff, and Managers). This abstract class provides

shared attributes like username and password but cannot be instantiated directly, serving only as a template for its subclasses.



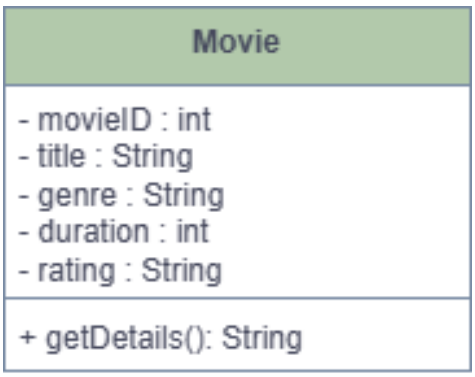
<u>Payment</u>

Similar to the Account class, Payment is defined as an abstract class to support various transaction methods. By using abstraction instead of a simple list, we allow the system to process different payment types, such as CreditCardPayment or PayPalPayment, using a unified interface. This ensures the system can adapt to new financial technologies without modifying the core booking logic.



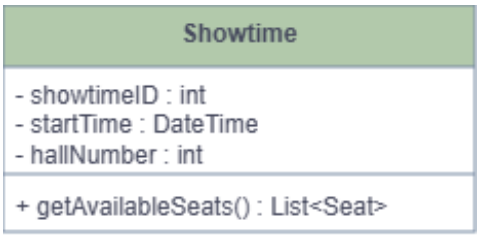
Movie

The Movie class serves as the central repository for film metadata. It stores essential information such as the title, genre, and duration. This class is associated with the Showtime class to provide users with specific details about the films they are browsing within the HappySeats interface.



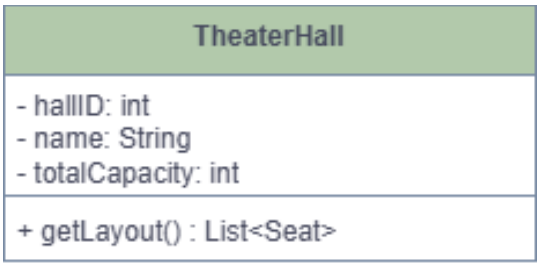
Showtime

The Showtime class represents a unique instance of a movie screening. It acts as the bridge between a Movie and a physical TheaterHall, tracking the specific date, time, and seat availability. It includes operations to query the real-time seat map to prevent double bookings.



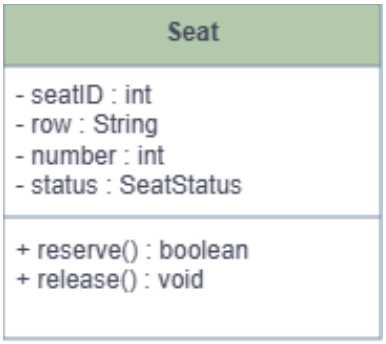
TheaterHall

The SeatSphereSystem maintains a composition relationship with the TheaterHall, which in turn maintains a composition relationship with the Seat class. This hierarchy ensures that if a theater hall is removed from the system, its specific configuration of seats is also removed, maintaining the integrity of the theater's physical inventory.



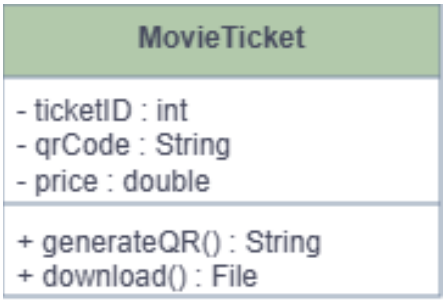
Seat

We define Seat as an abstract class to represent physical chairs within an auditorium. This allows the system to scale and handle different seat types, such as StandardSeat or PremiumSeat, each with its own pricing or amenity attributes. The status of each seat is tracked using enumeration to ensure accurate real-time viewing for the customer.



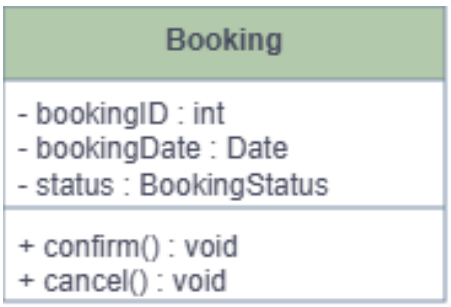
MovieTicket

The MovieTicket class is the digital deliverable generated upon a successful transaction. It contains unique identifiers such as a QR Code for theater staff to validate. This class is dependent on both the Booking and Showtime classes to ensure the ticket correctly reflects the customer's purchase.



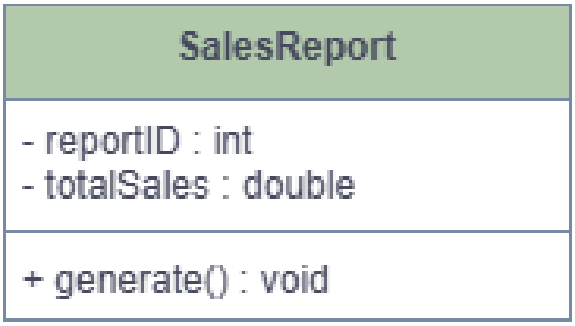
Booking

The Booking class manages the transactional state of a user's reservation. it records the booking date, total price, and the current status of the order (e.g., Pending, Confirmed, or Cancelled). It coordinates between the User account and the Payment class to finalize the seat inventory update.



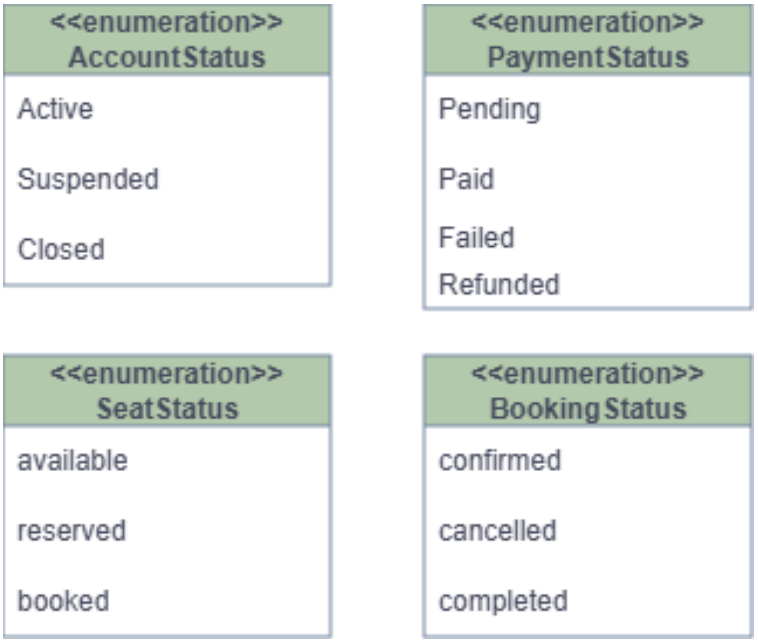
SalesReport

To assist theater managers in monitoring performance, the SalesReport class aggregates data from all completed bookings. It provides operations to generate financial summaries and attendance trends. This class is essential for stakeholders to track revenue and make data-driven decisions regarding movie scheduling and theater hall assignments.



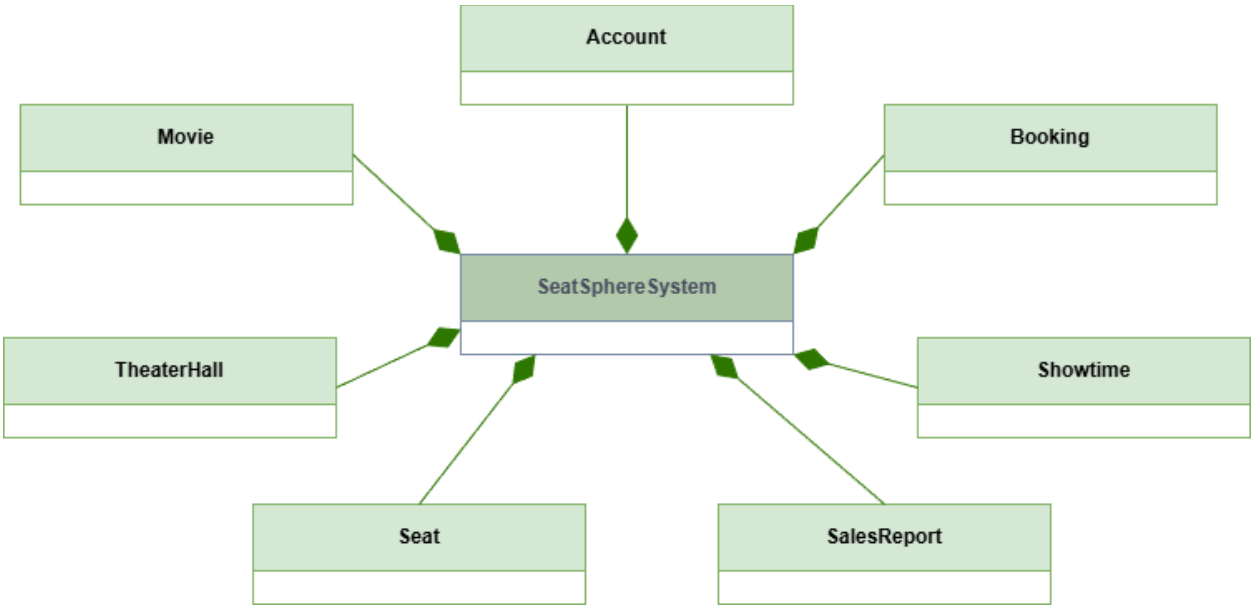
Enumerations and Custom Data Types

- Tracks the real-time availability of seats to prevent double-booking
- Manages user access rights.
- Tracks the lifecycle of a ticket purchase.



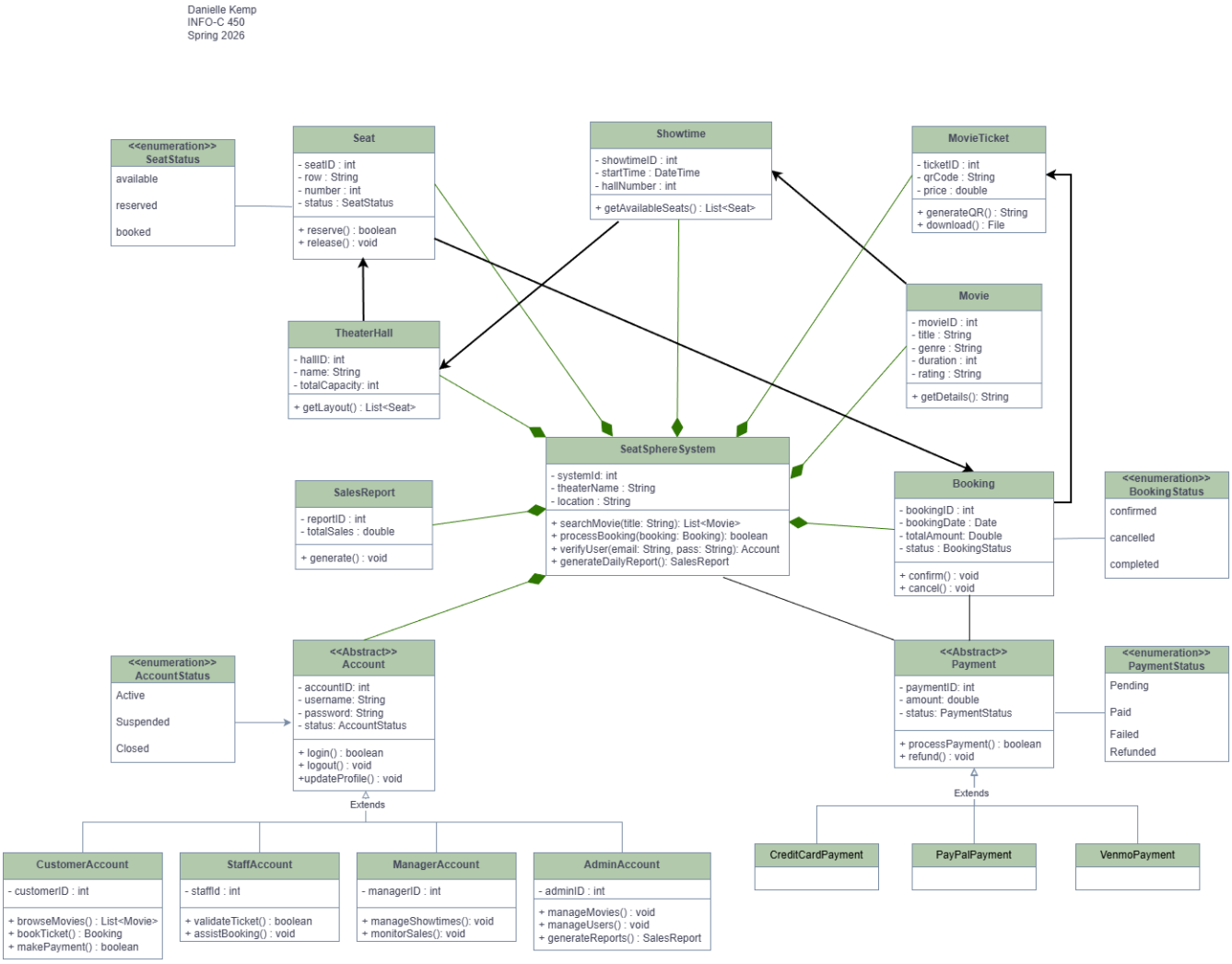
Composition

The SeatSphereSystem class includes Movie, Account, Booking, Showtime, TheaterHall, Seat, SalesReport.



Class Diagram

The final diagram comes out like this. It shows the responsible lifecycle.



System Sequence Diagram and Activity Diagram

System Sequence Diagrams

SSD1:

The Movie Selection, Seat Reservation, and Payment Processing are chosen to show the sequence diagram example for the SeatSphereSystem.

- Actor: Customer
- System: SeatSphereSystem
- Objects: SeatSphereSystem, PaymentProvider

Steps:

1) The customer selects a specific movie and showtime.

2) The system displays the movie details and a map of available seats.

3) The customer selects their preferred seats and requests to book.

4) The system calculates the total amount (ticket price + fees) and prompts for payment details.

5) The customer provides payment information (e.g., Credit Card or PayPal, Venmo).

6) The system sends the transaction details to the Payment Provider.

7) The Payment Provider processes the transaction and returns a success or failure status.

8) If the payment is successful: * 1. The system generates booking confirmation.

- The system creates a QR code ticket for the customer to download.

9) If the payment is unsuccessful: * 1. The customer sees an error message (e.g., "Insufficient Funds" or "Payment Declined").

- The system prompts the customer to try a different payment method.

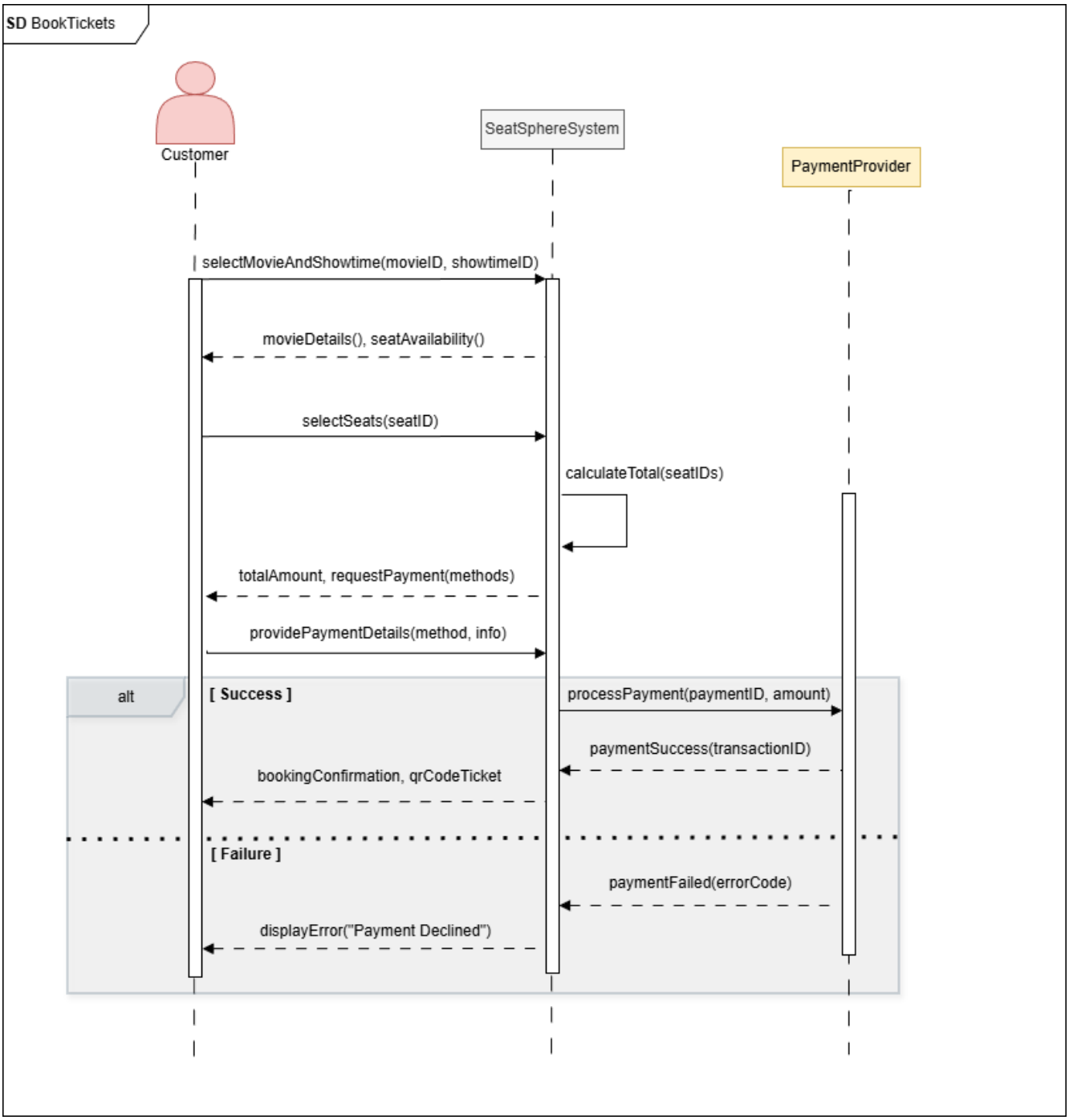


Figure 2: SeatSphere System Sequence Diagram 1
Designed by Danielle Kemp via draw.io

SSD2:

The TheaterStaff, MovieTicket, and SeatSphereSystem are chosen to show the sequence diagram example for the SeatSphereSystem.

- Actor: Theater Staff
- Objects: SeatSphereSystem, MovieTicket

Steps:

1) The theater staff scans the QR code from the customer's mobile or printed ticket.
2) The system requests the ticket details and current status from the database.
3) The system identifies the ticket status (e.g., Confirmed, Used, or Expired).
4) The system performs an internal check to verify the ticket matches the current movie and showtime.
5) **If the ticket is valid:**

- The system displays a "Valid" confirmation and the customer's seat number.
- The theater staff confirms the customer has entered the hall.
- The system updates the ticket status to "Completed."

6) **If the ticket is unsuccessful (Invalid):**

- The system displays an error message (e.g., "Ticket Already Scanned" or "Invalid Showtime").
- The system notifies the staff to deny entry.

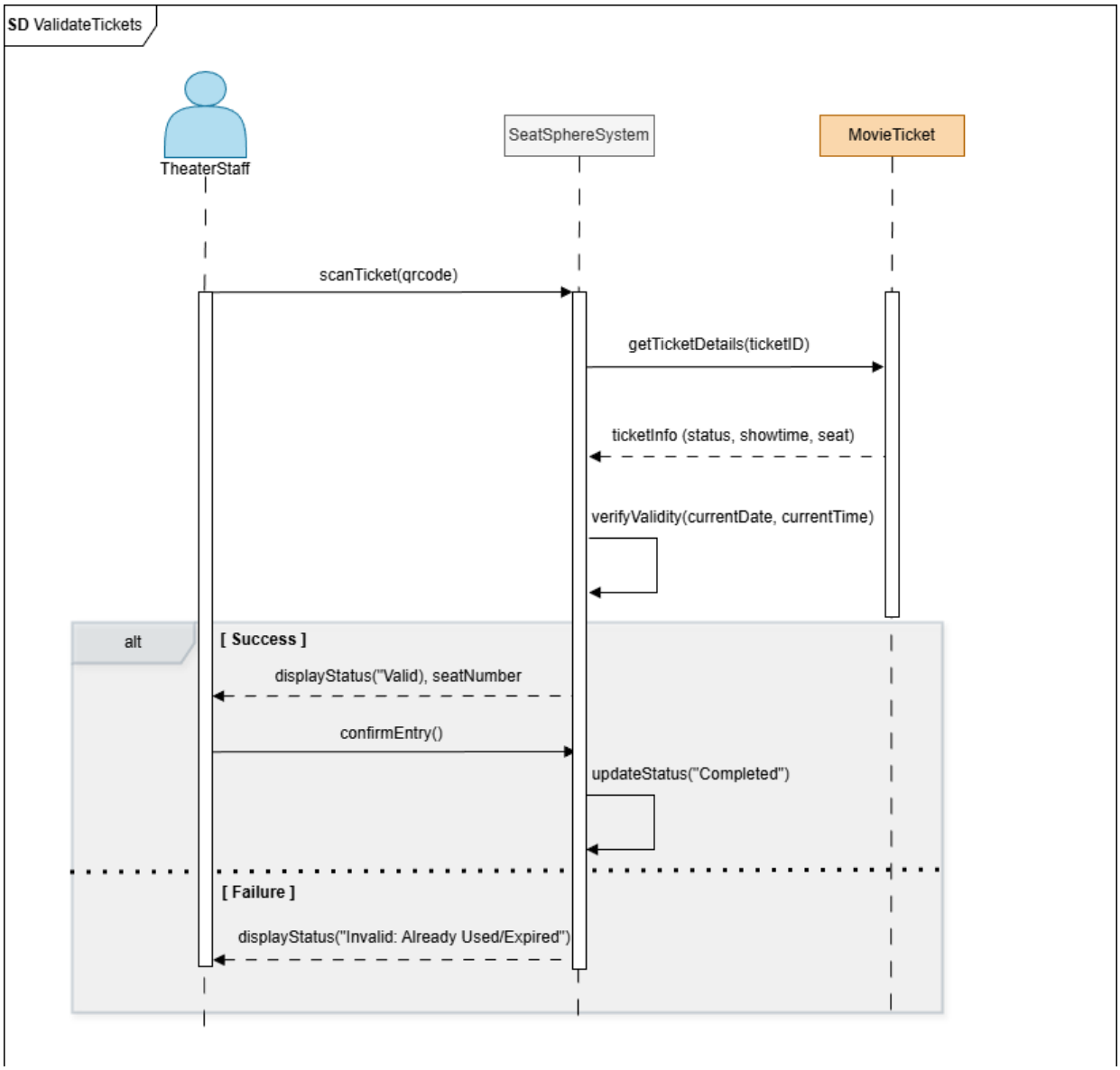


Figure 3: SeatSphere System Sequence Diagram 2
Designed by Danielle Kemp via draw.io

Activity Diagrams

Activity Diagram 1

- Initial State: The customer selects the movie and/or the showtime from SeatSphere.
- Final State: System generates digital QR ticket and customer receives the booking confirmation.

Actions:

The customer selects the movie and/or the showtime from the SeatSphere application. The system checks the database for seat availability to see if the seats are available. The system will display a seating map and allow the customer to select specific seats. Once the seats are chosen, the system will calculate the total price including taxes and fees, then the system will prompt the customer to choose their payment provider and enter their payment information. Once the payment details are received, the system will communicate to the payment provider to authorize the transaction. If the payment is successful, the system will then update the seat status to “Reserved” and generate a unique QR code ticket. The system will display a confirmation message to the customer.

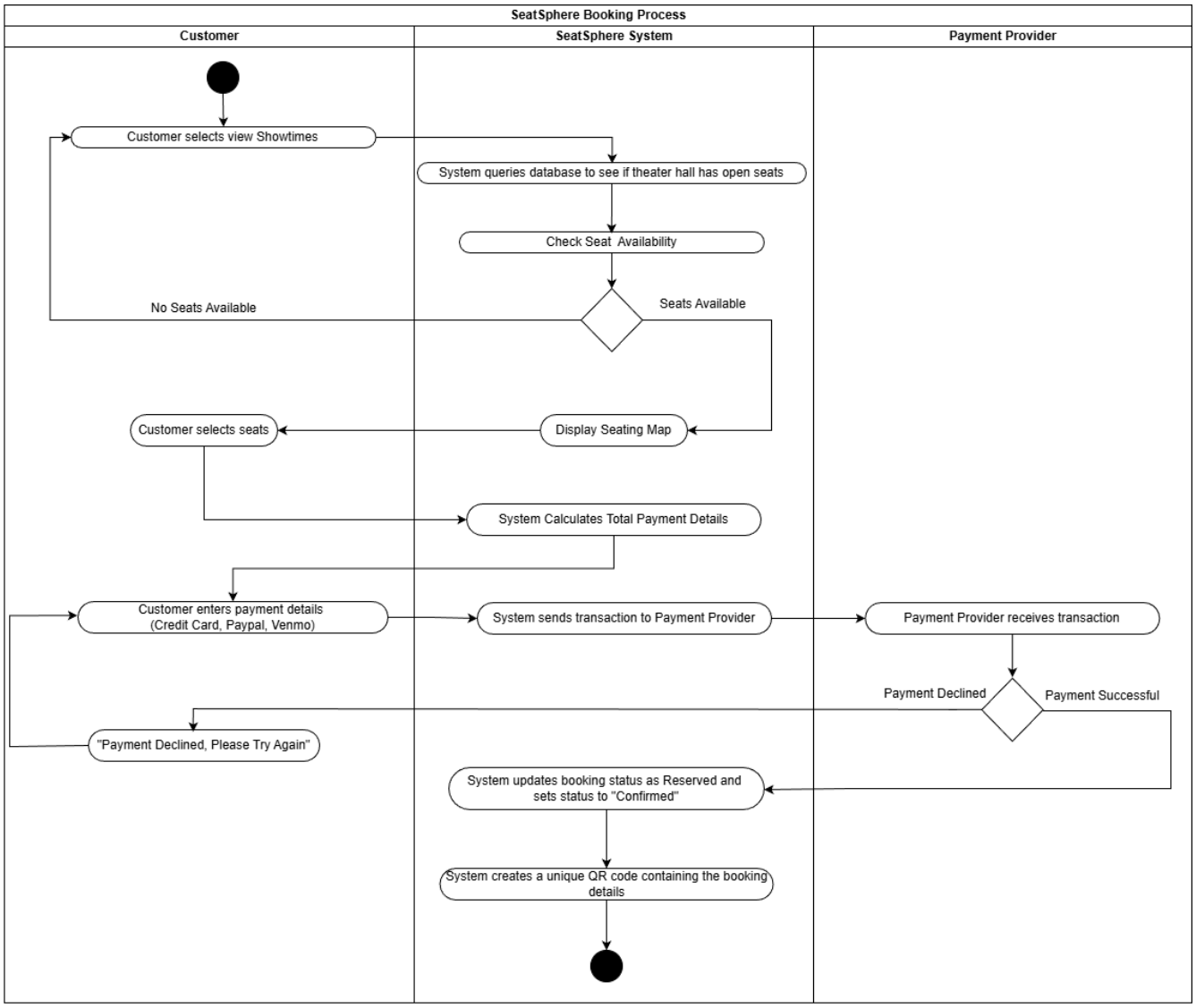


Figure 4: Activity Diagram 1
Designed by Danielle Kemp via draw.io

Activity Diagram 2

- Initial State: Theater staff will activate the scanning interface on the mobile device.
- Final State: The customer is admitted into the theater hall and the ticket status is updated.

Actions:

The theater staff member scans the customer's QR code using the system's scanning tool. The system will retrieve the ticket data and verify the validity against the current date, movie date, movie time, theater hall, and seat. If the ticket reads as valid, the system will display an "Valid" and display the customers seat number and theater number, which will allow the staff to confirm to the customer their entry. Upon confirmation, the system updates the ticket status in the database to "Completed" and this will mark the ticket as unusable. If the ticket is invalid, then the system displays an error message, "Invalid: Already Used/Expired" and the theater staff denies the entry request.

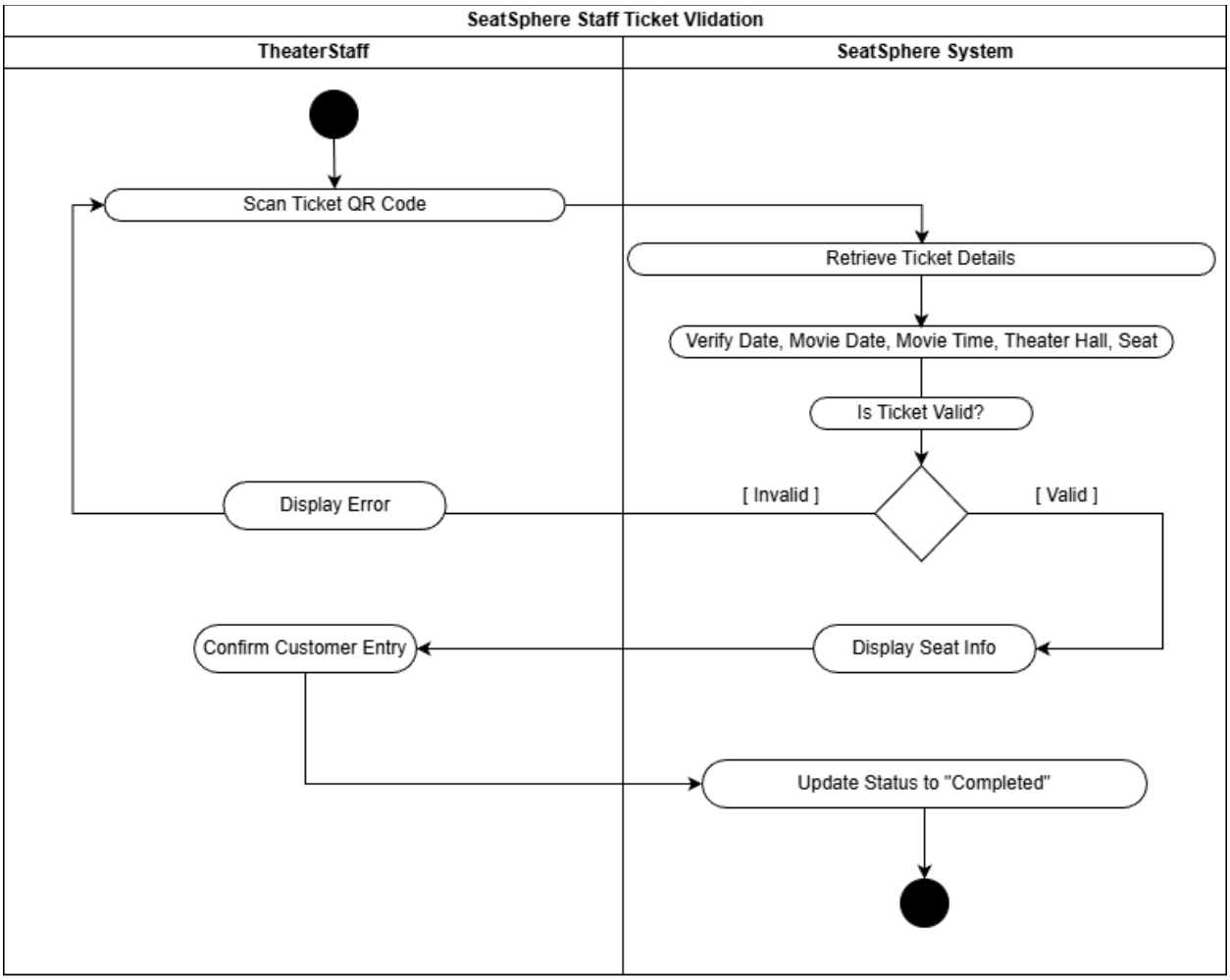
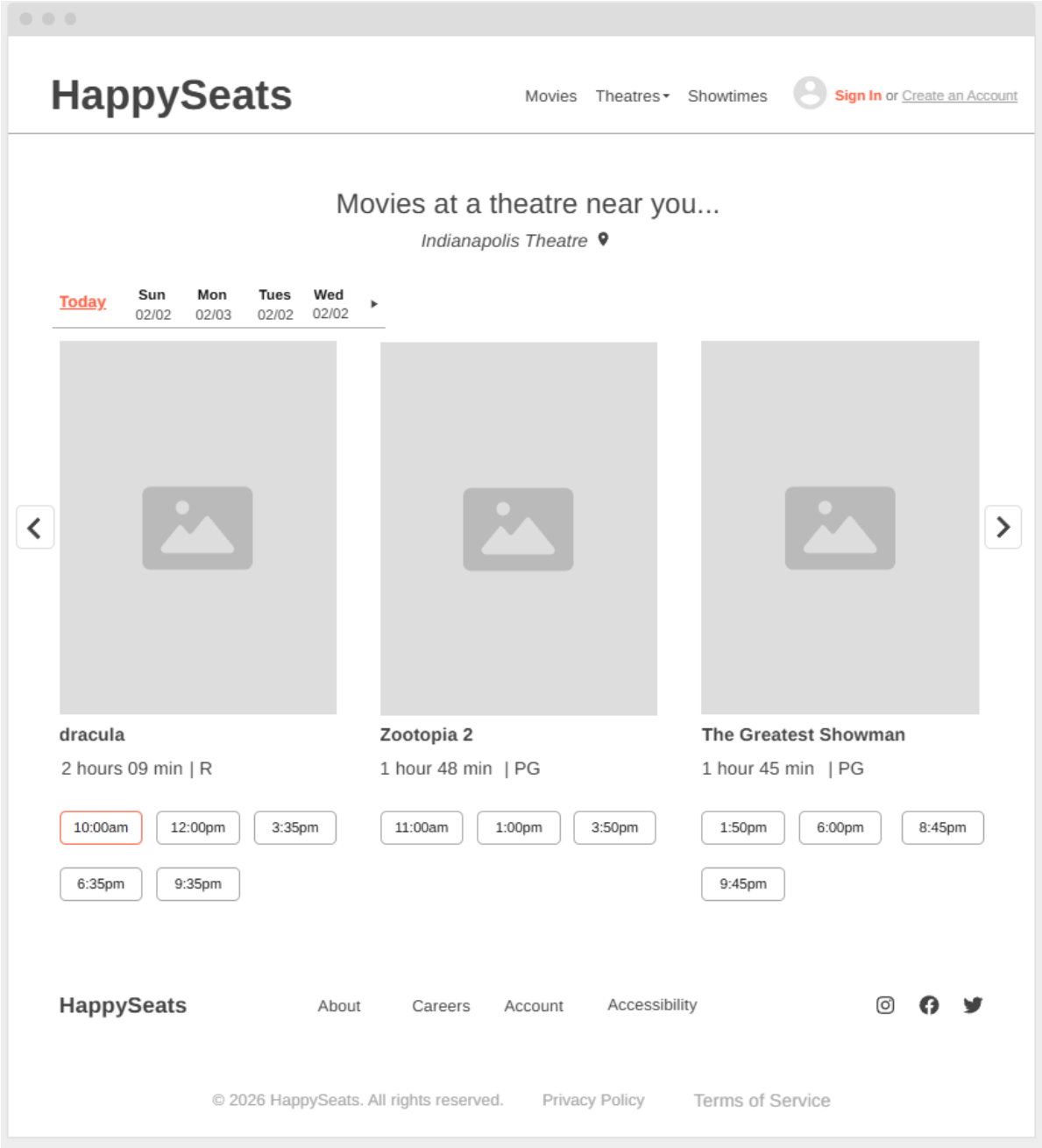


Figure 5: Activity Diagram 2
Designed by Danielle Kemp via draw.io

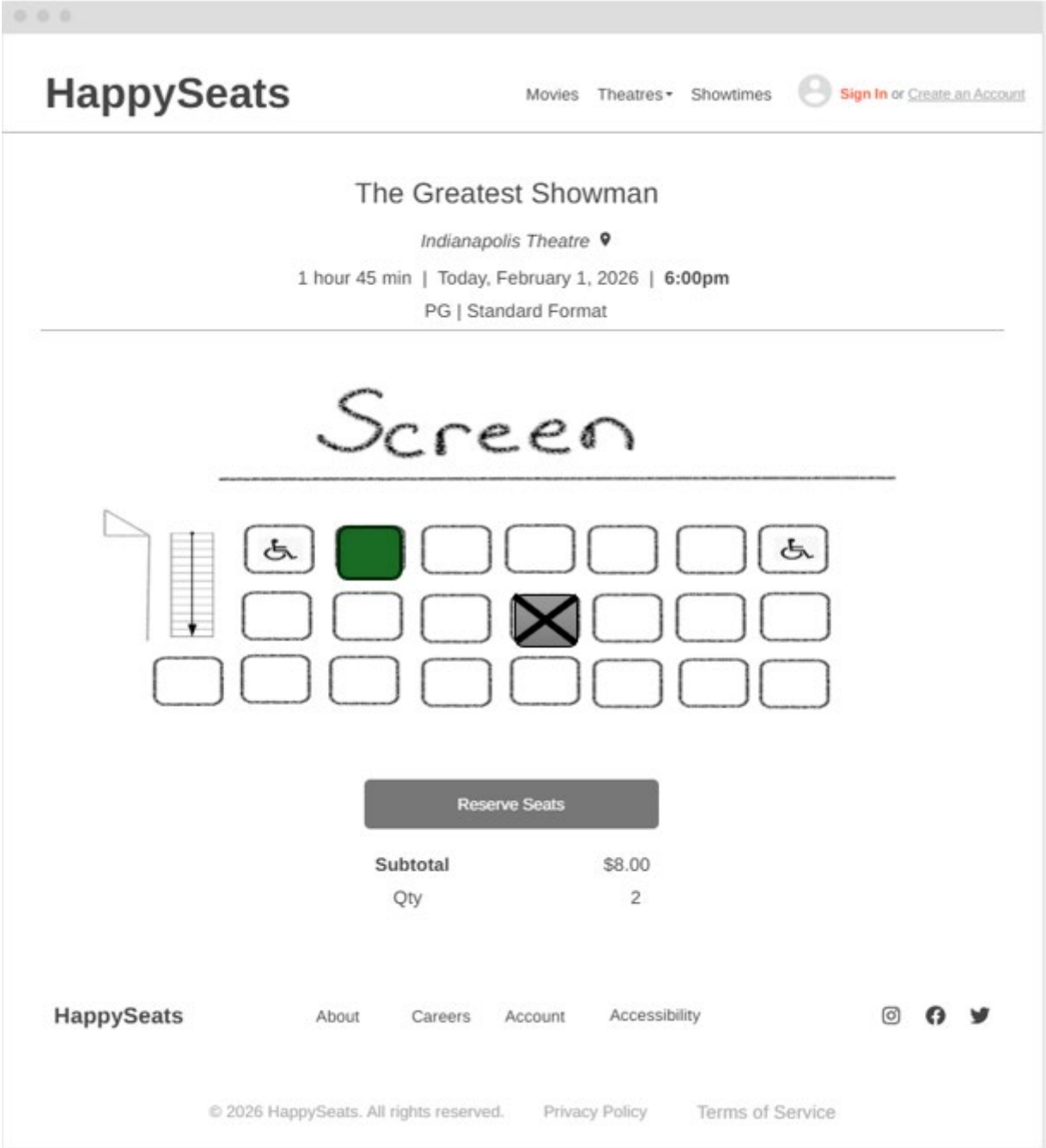
User Interface Specification

Preliminary Design

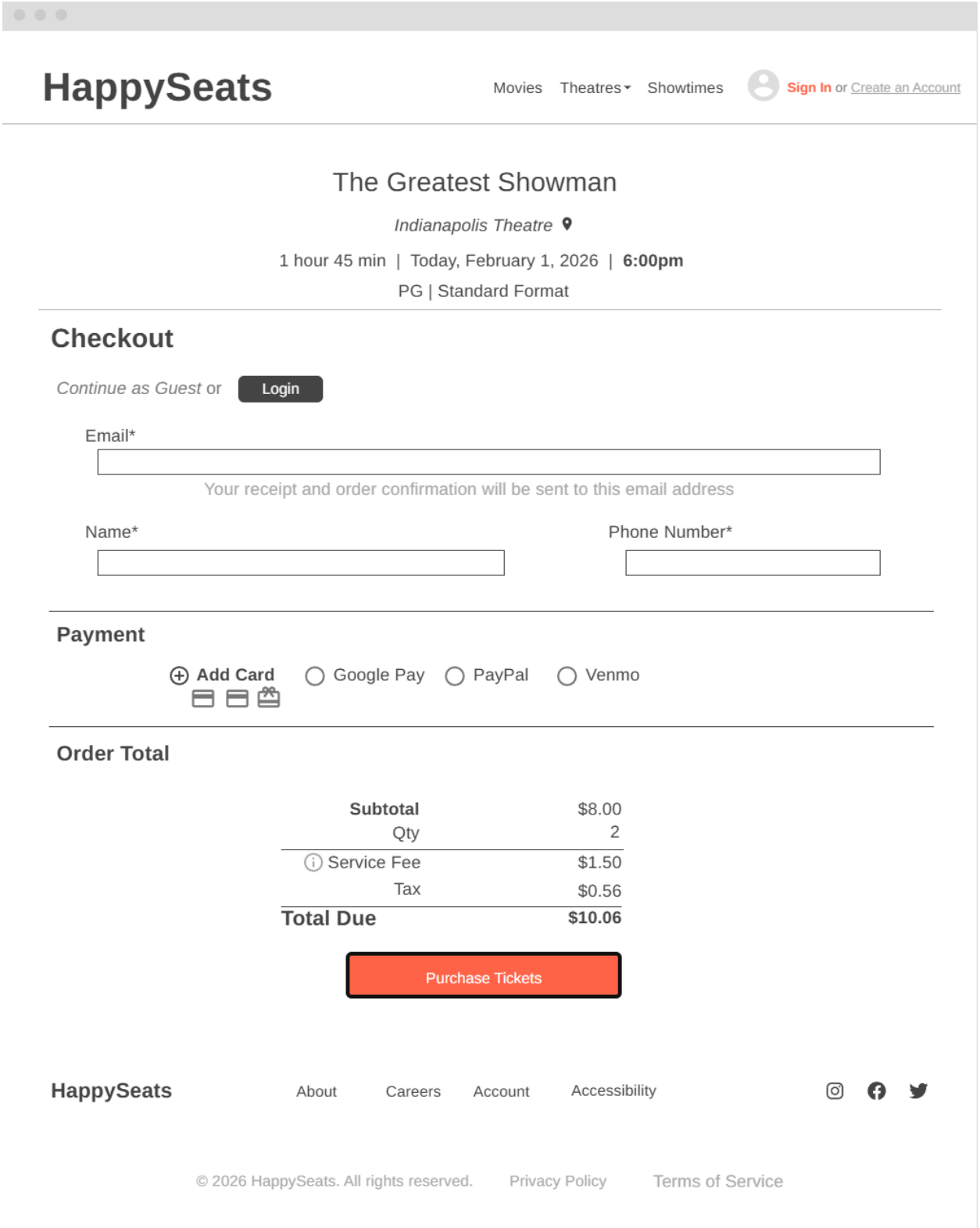
Use Case 1: Customer selects seats and confirms a booking



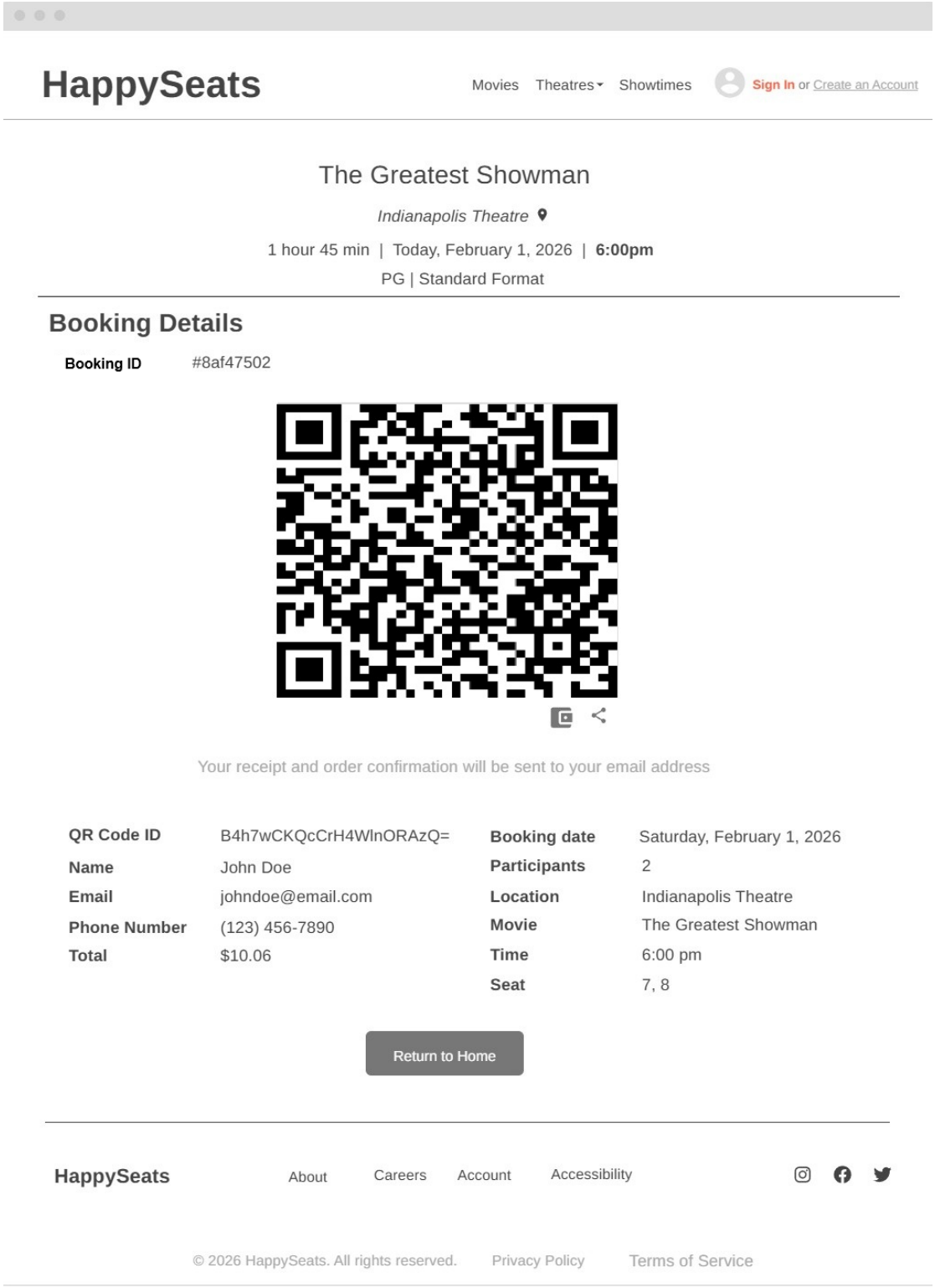
Movie and Showtime Selection: Customer chooses movie and showtime. Once showtime is selected it brings customers to next screen.



Customer selects from available seats and clicks reserve seats.

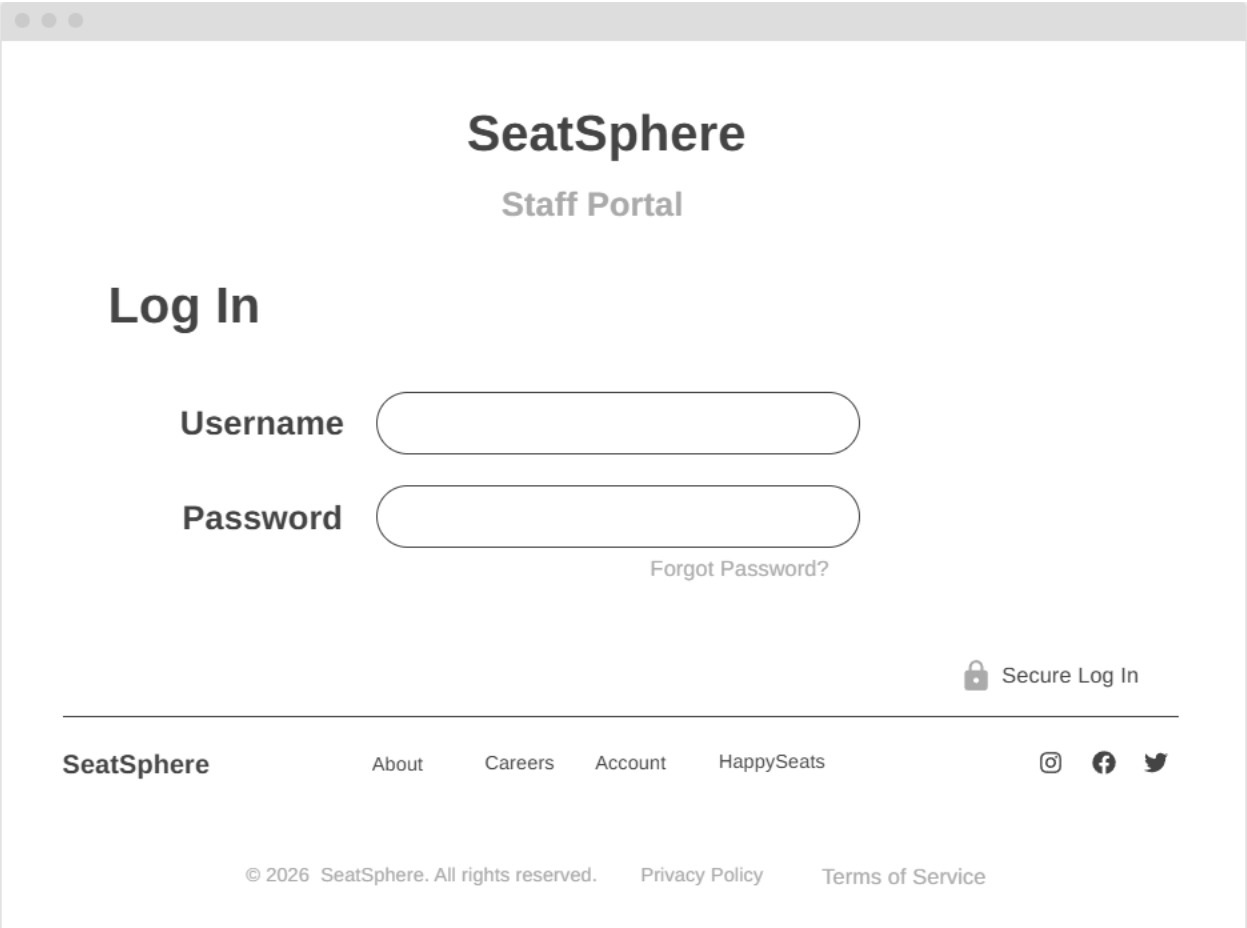


Payment page (System shows 2 seats x $4.00 = $8.00) Customer inserts credentials and payment information then clicks Purchase Tickets.

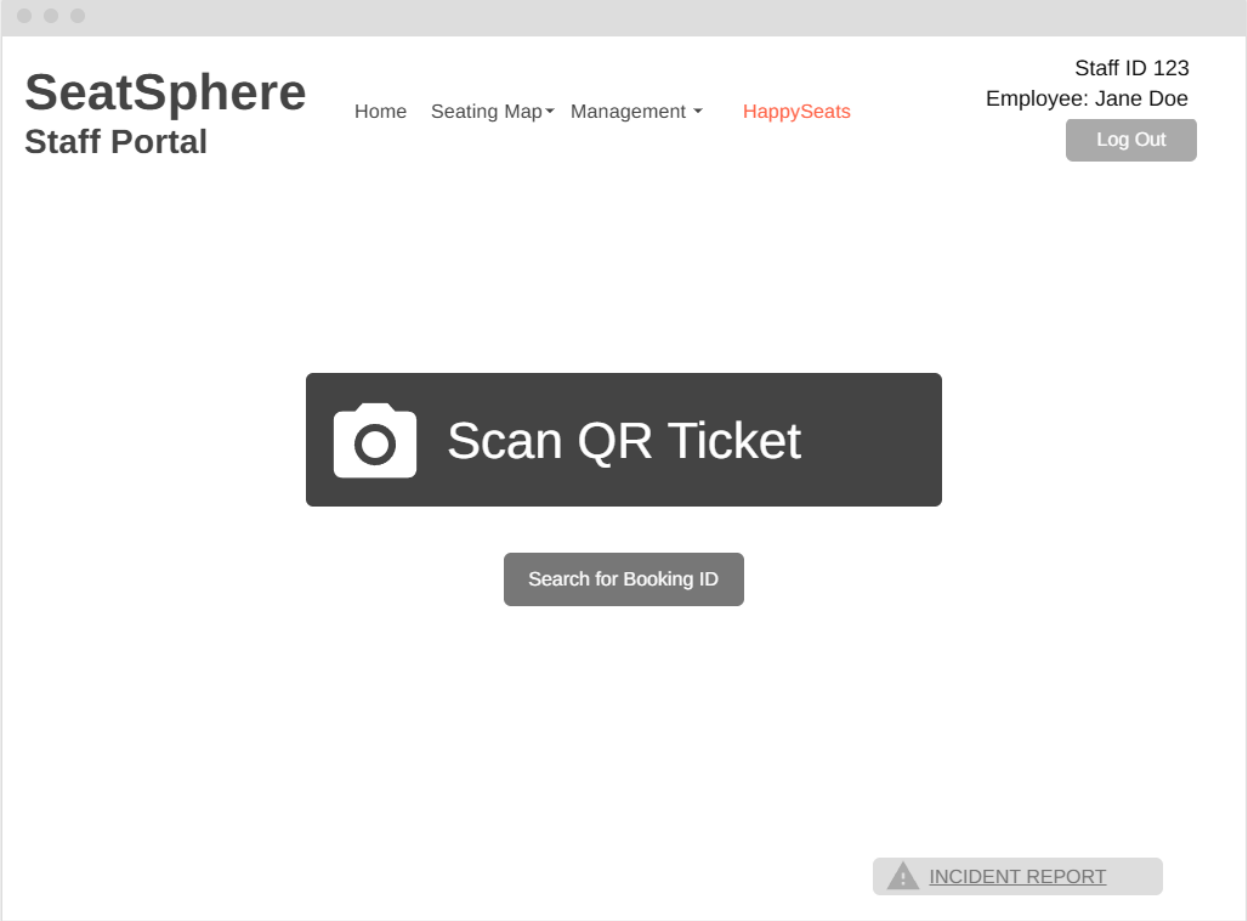


Confirmation page displaying QR code and confirmation of order details to Customer. Customer can click ‘Return to Home’ button which system will loop them back to the home page.

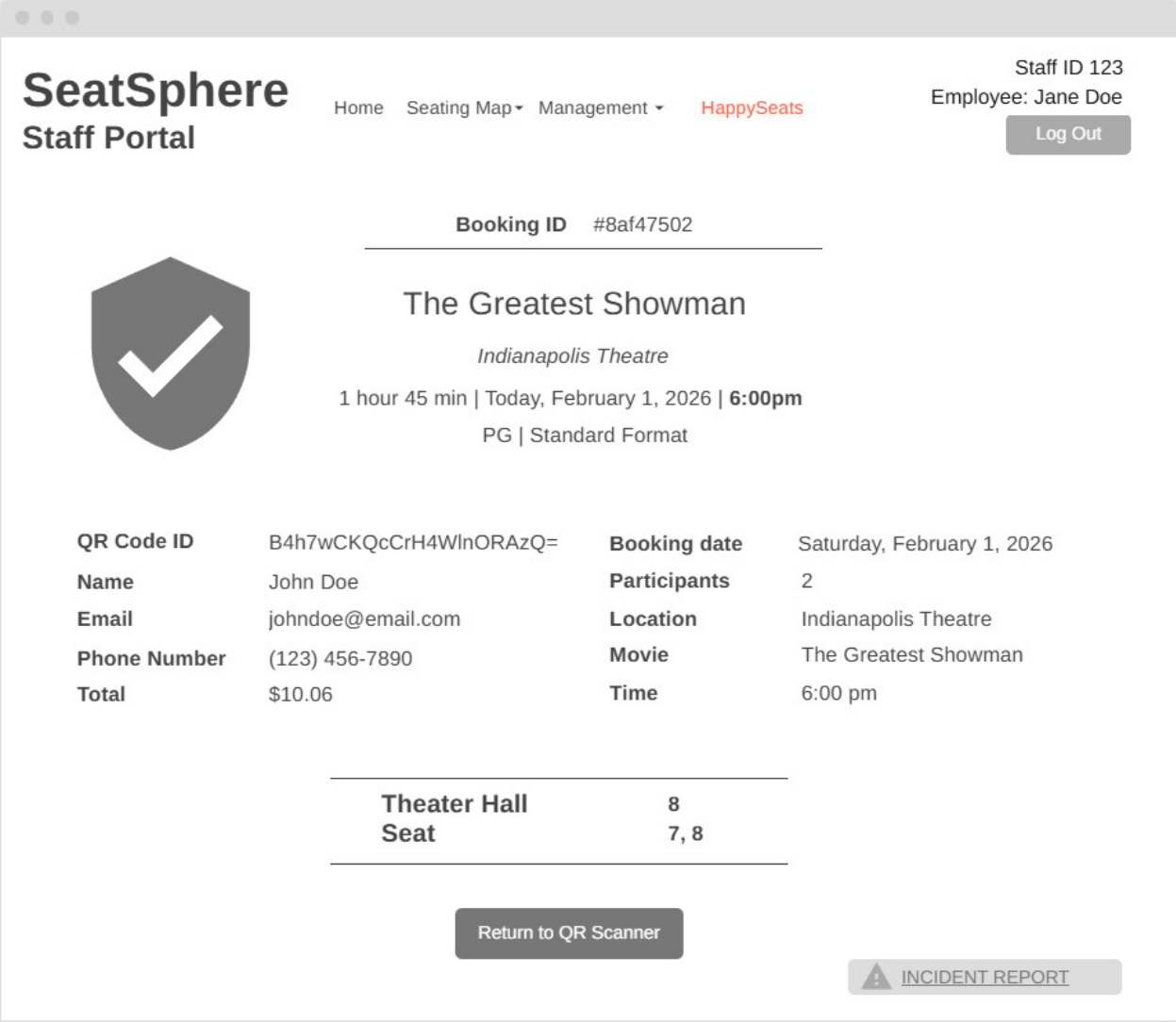
Use Case 2: Staff scans and validates ticket



Log in page for Staff to enter log in credentials.

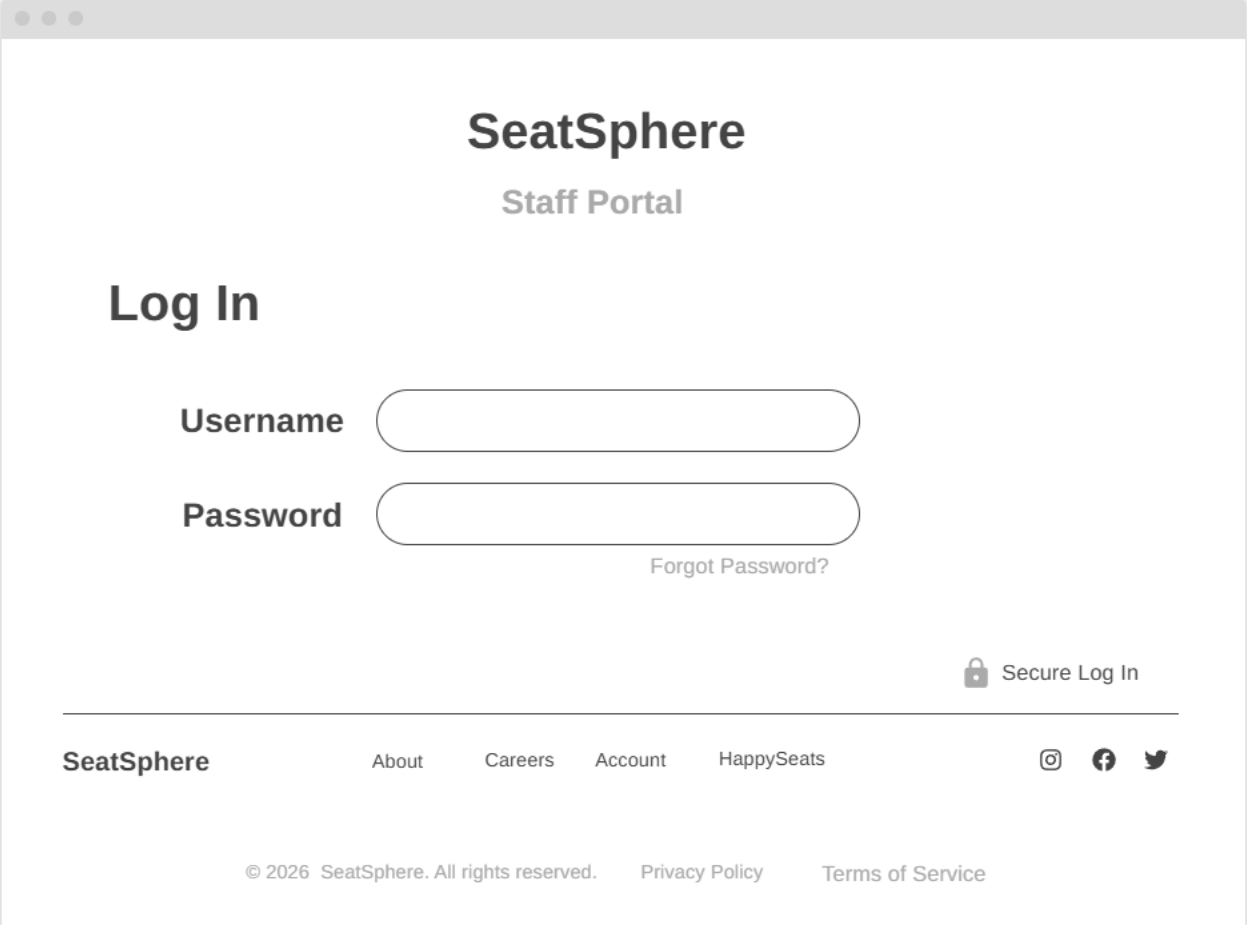


Theater staff clicks 'Scan QR Ticket' and scans QR code. If QR Ticket unavailable, Theater Staff have the option to click the button 'Search for Booking ID' to manually enter Booking ID number.

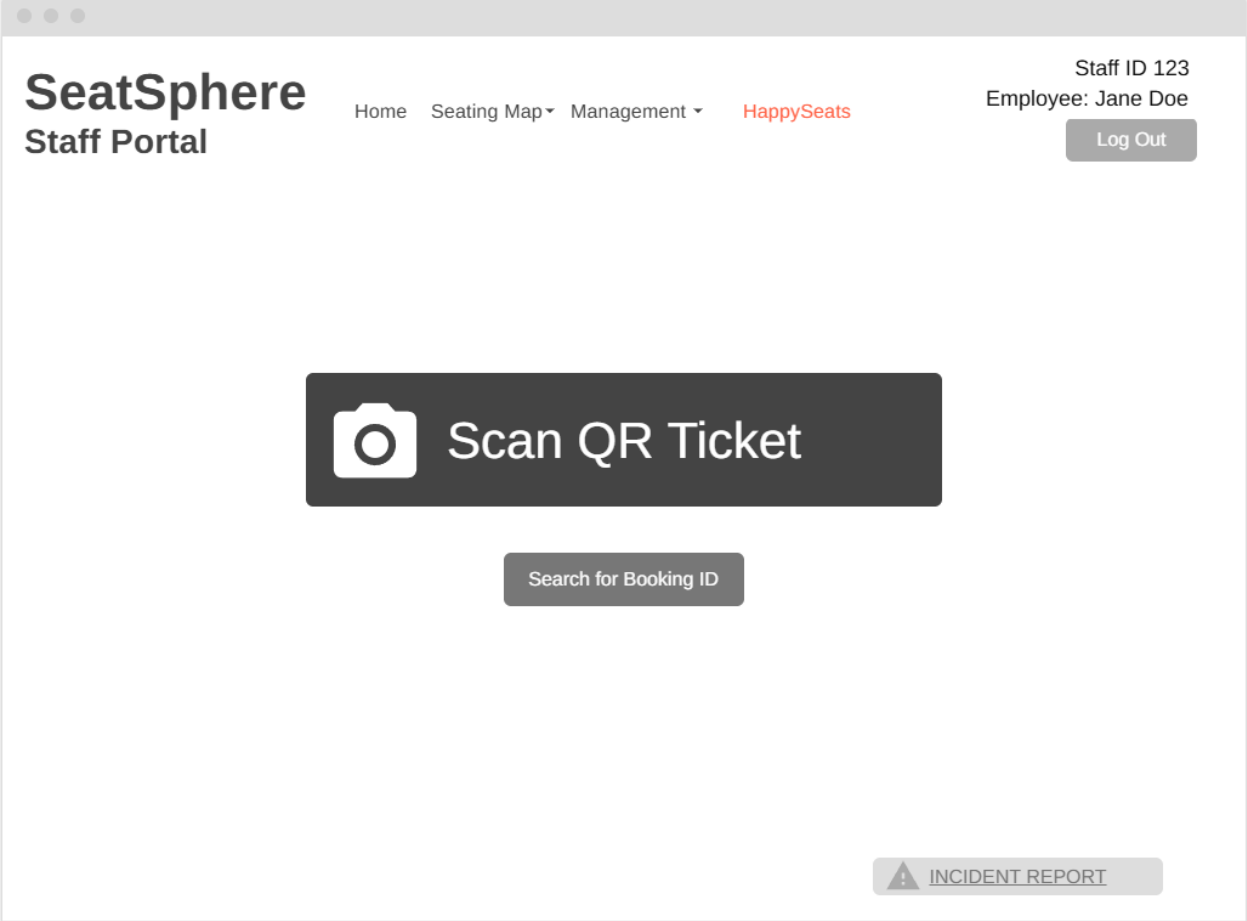


Theater Staff has valid checkmark display on screen confirming customers movie ticket is valid. It also displays the QR information and customer information showing the stored information from the database.

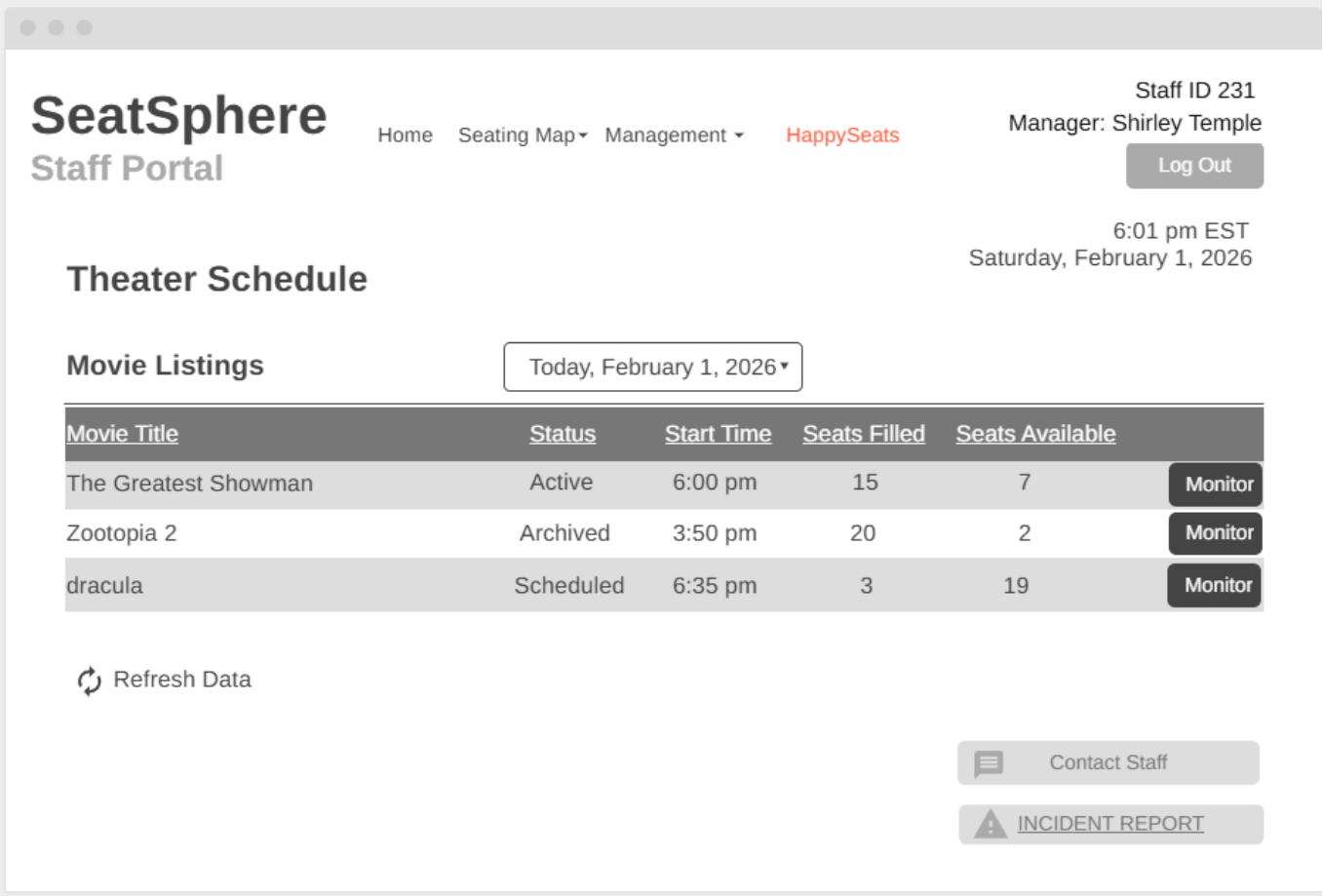
Use Case 3: Theater Manager monitors Seat Availability



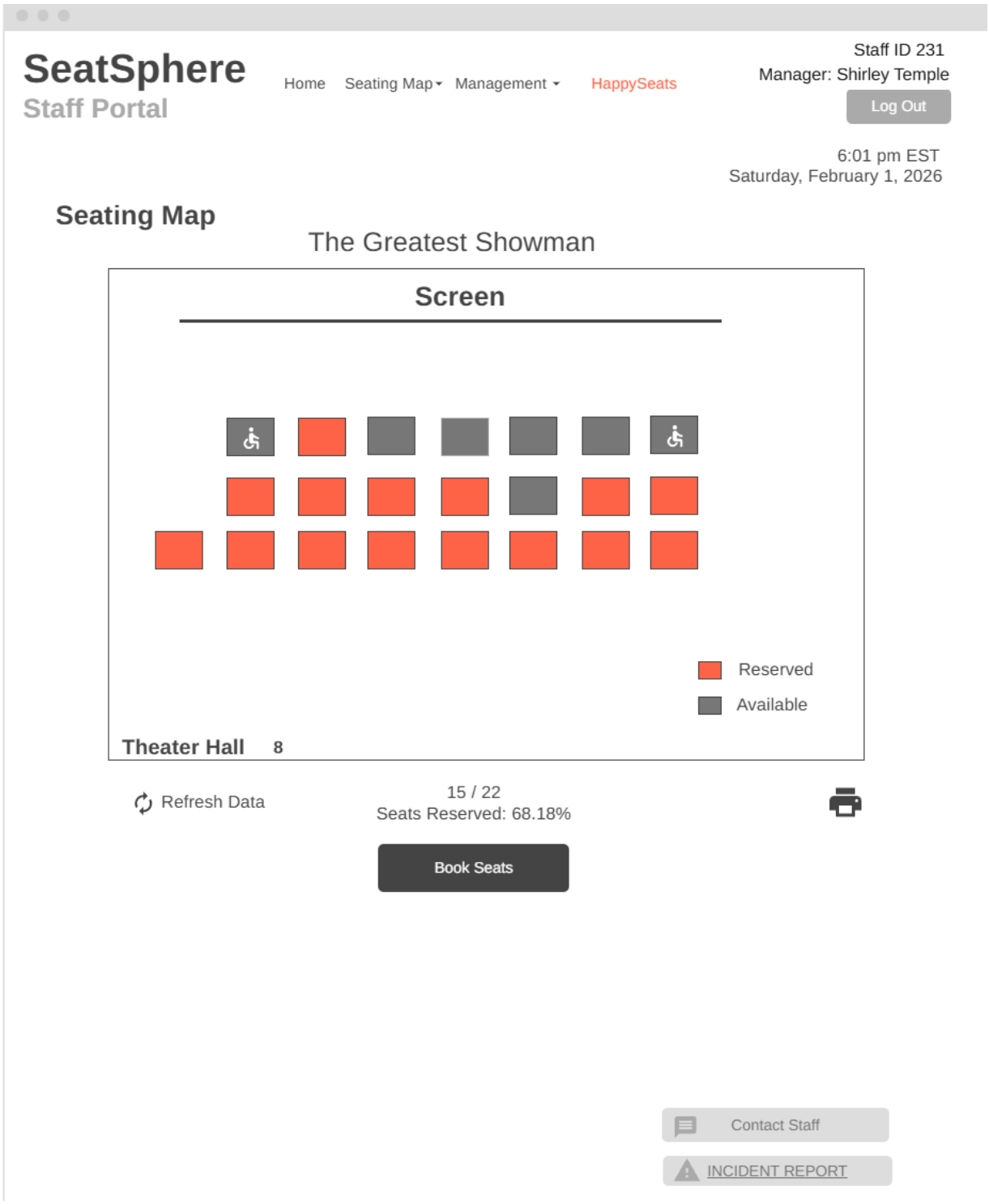
Log in page for Staff to enter log in credentials.



Primary home page for all staff and managers. Log in is at the top right corner. In this example you can see the employee is logged in so the manager would need to log out and log in.

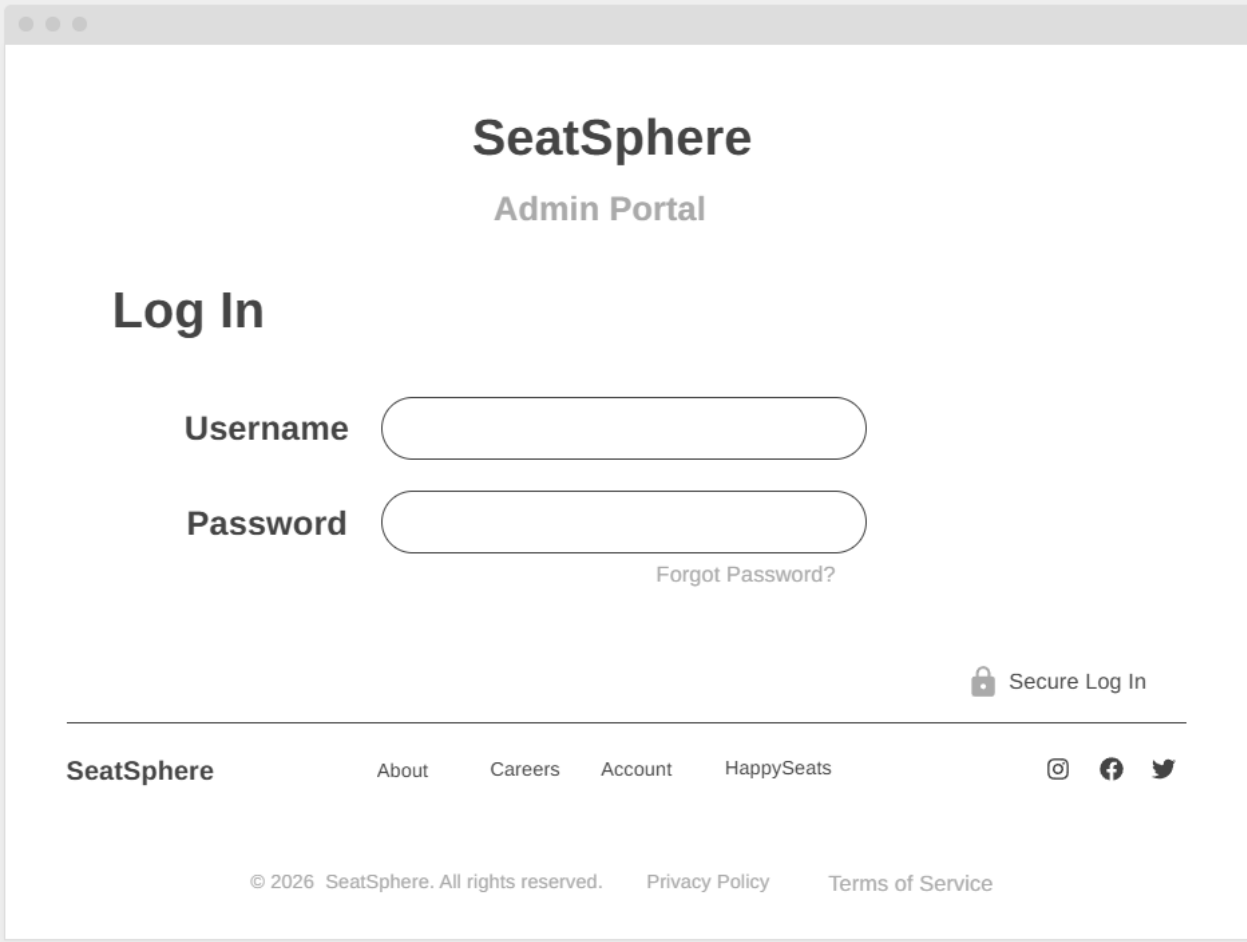


Theater Manager is able to look over theater schedule (under the Management drop down in navigation bar). Theater Manager will click on Monitor to then open up the Live Seating Map.

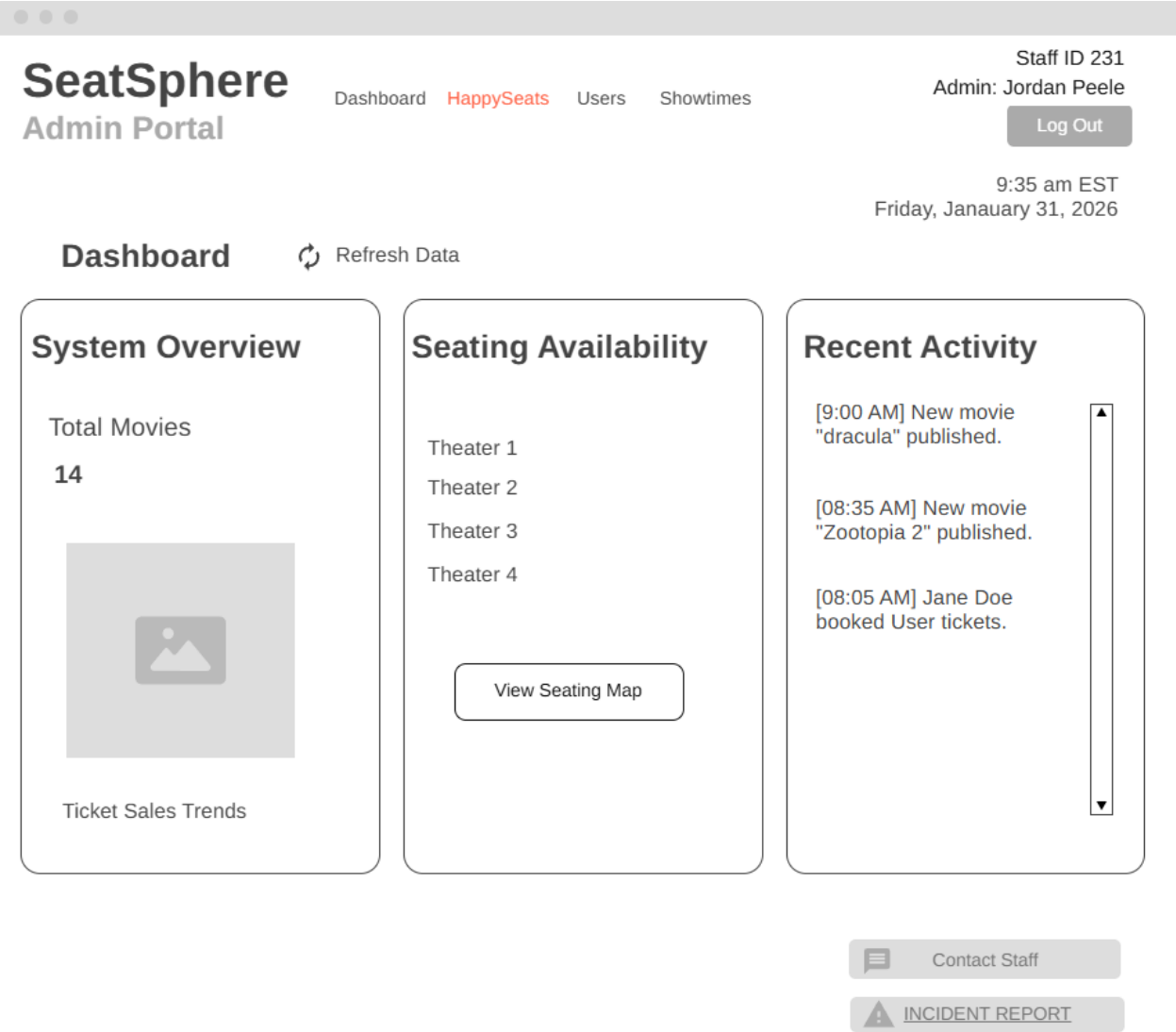


Theater Staff is able to view the Seat Map and see the seating reservation for the theater hall and the current movie playing.

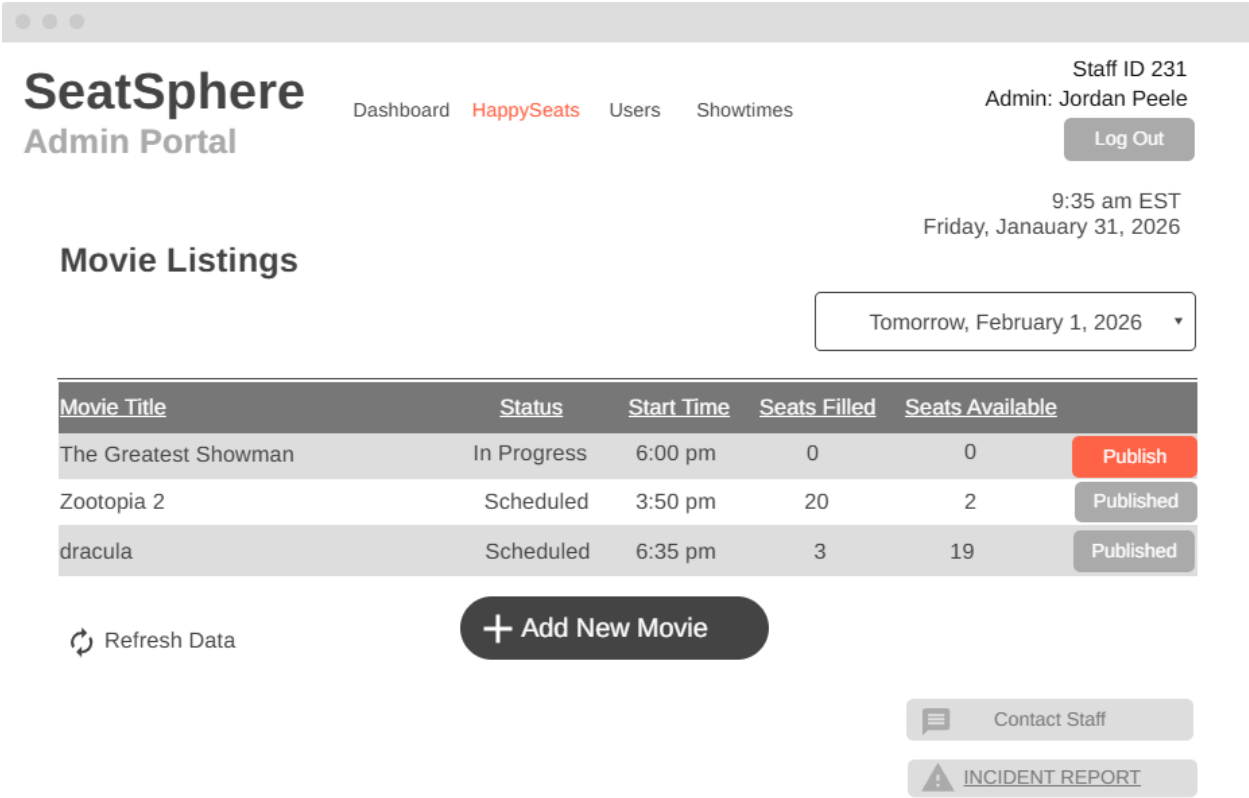
Use Case 4: Admin adds a new movie listing



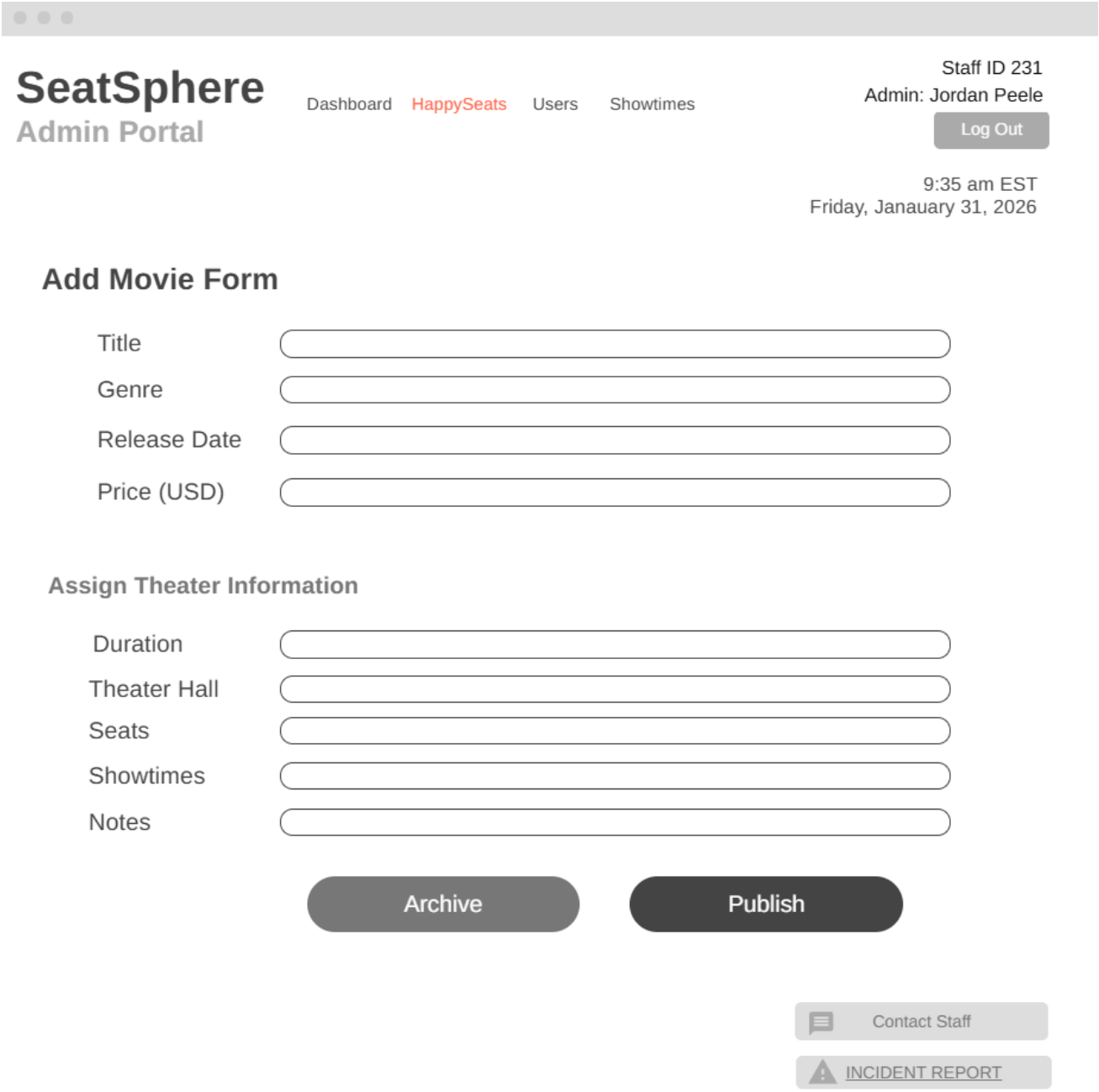
Admin logs into the Admin Portal. Similar to the staff portal but noted at the top that it is Admin Portal.



Once Admin has logged in, they arrive at the dashboard / home page. From here they can navigate to the Movie Listings under the hover over drop down button, ‘Dashboard’ or click on System Overview which will take the Admin to the Movie Listings page.

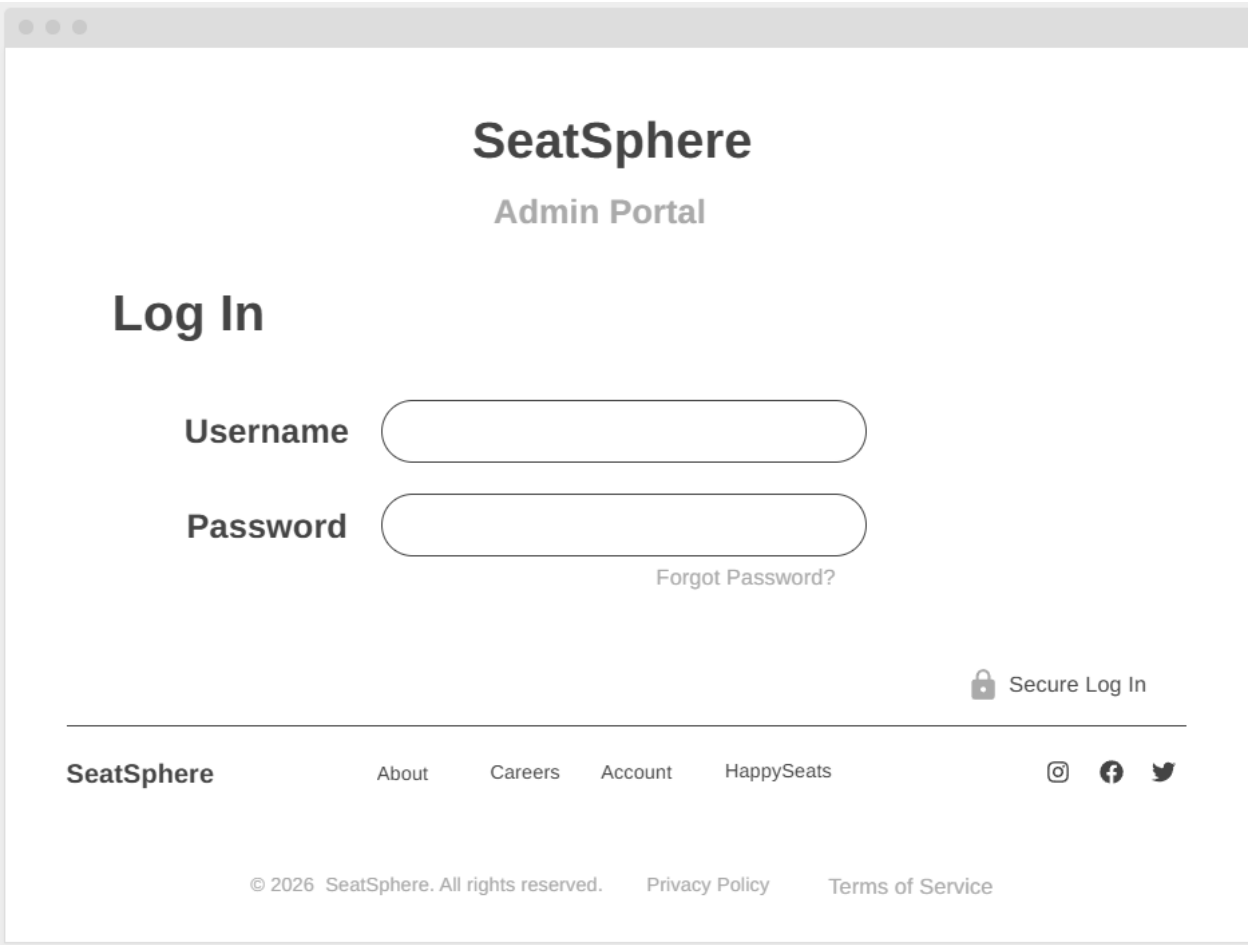


Move Listings page will allow the Admin to add new movies, publish, or see if a movie is published/active.

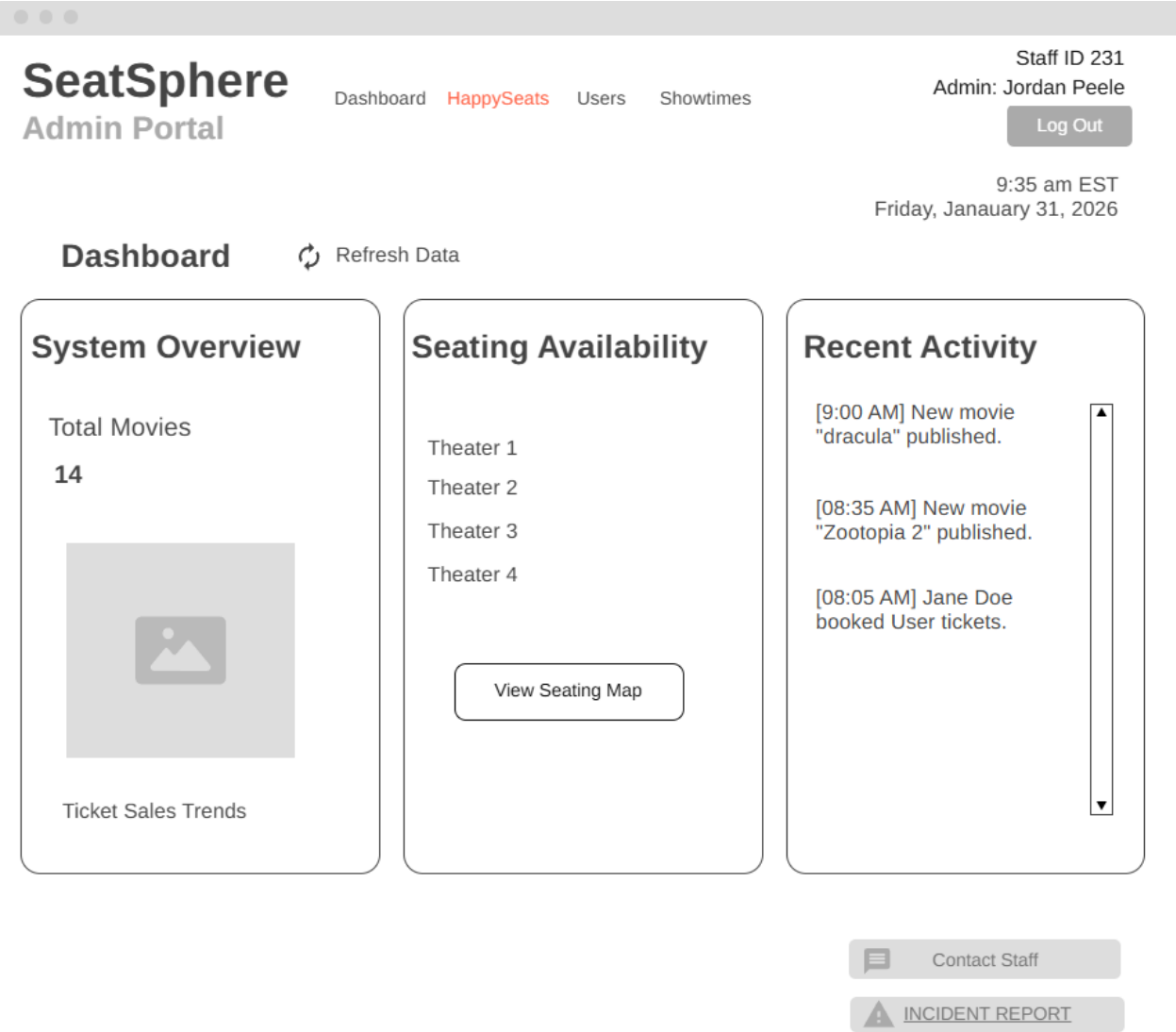


The Admin can add new movies to the Add Movie Form page. Once information about the movie is entered, the Admin can either Archive (to work on later) or click Publish. If Archive or Publish is clicked, it will bring them back to the Movie Listings page where they can see the status of the movie listing.

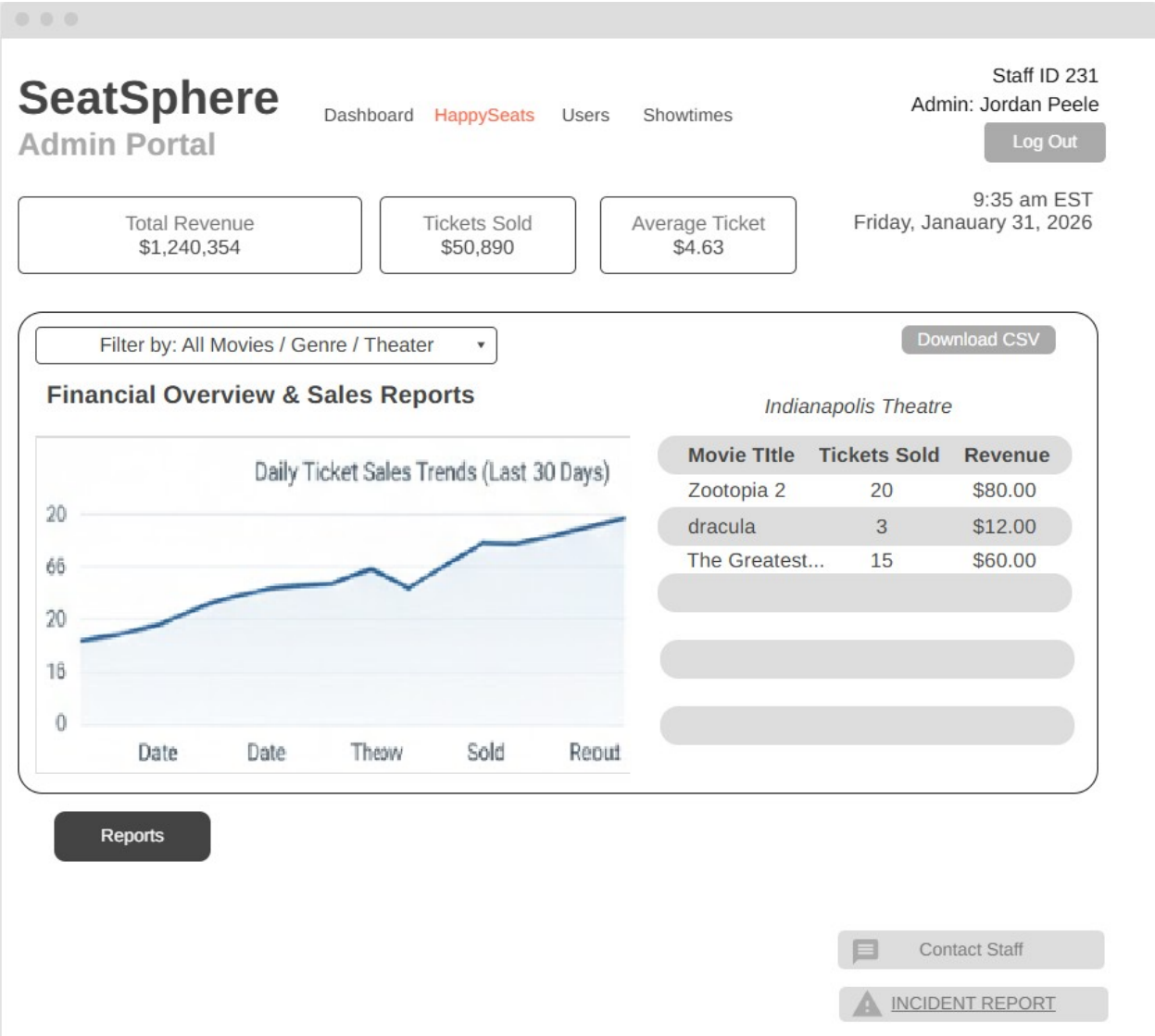
Use Case 5: Admin Views Sales Report



Admin logs into the Admin Portal. Similar to the staff portal but noted at the top that it is Admin Portal.



Once Admin has logged in, they arrive at the dashboard / home page. From here they can navigate to the 'Ticket Sales Trends' under the System Overview section on the Dashboard screen.



Admin lands on the Ticket Sales Trends page which displays the Financial Overview & Sales Reports. Varius drop downs are available to switch between theaters, movies, and genres to look at various reports. An available Download CSV button is clickable for the Admin to download reports.

User Effort Estimation

Usage Scenario	**Navigation Path**	**Clicks**	**Keystrokes**
Customer books ticket	Home page/Movies → Seat Selection→ Pay→ Confirmation Page	9	<100
Staff scans and validates ticket	Log In → QR Scan Page → Scan QR Code →Valid QR Page	4	0
Theater Manager monitors seat availability	Log In → Theater Schedule → Seating Map	5	0
Admin adds new movie listing	Log In → Dashboard → Movie Listings →Add Movie Form	5	>100
Admin views sales report	Log In → Dashboard → View Sales Report	2	0

*Key stroke totals will **exclude** the log in page.*

Traceability Matrix

System Requirements

No.	Priority Weight	Description
REQ 1	5	Users must be able to browse available movies and view ratings and showtimes.
REQ 2	5	The system must allow users to view a visual seat map and select specific seats.
REQ 3	4	Customers must be able to securely purchase tickets and receive confirmation.
REQ 4	5	Administrators must be able to add, edit, or remove movie listings.
REQ 5	3	Staff must be able to monitor real-time seat occupancy and availability.
REQ 6	4	Administrators must be able to generate and view financial sales reports.
REQ 7	5	The system must support staff/admin login for portal access and security.
REQ 8	3	The system shall implement a database lock on seats during the checkout process.
REQ 9	2	The system should allow administrators to export reports in CSV format.
REQ 10	5	Staff must be able to validate customer tickets via QR code scanning.
REQ 11	4	The seat selection map must update within 2 seconds of a booking to ensure real-time accuracy.

REQ 12	2	The UI must follow ADA standards, providing high-contrast color options for the seating map.

Use Cases

No.	Description
UC 1	Browse & Purchase Tickets: A customer browses the movie grid, selects a film, and completes a transaction.
UC 2	Interactive Seat Selection: A customer views a live theater map and selects seats based on availability.
UC 3	Manage Movie Content: An administrator uses a form-based interface to publish new movie listings.
UC 4	Monitor Theater Hall: A manager views the real-time seating ratio (e.g., 15/22 reserved) to oversee the house.
UC 5	Generate Financial Reports: An administrator views revenue trends and exports sales data via CSV.

Traceability Matrix

Req't	PW	UC1	UC2	UC3	UC4	UC5
REQ1	5	X				
REQ2	5		X			
REQ3	4	X				
REQ4	5			X		
REQ5	3				X	
REQ6	4					X
REQ7	5			X	X	X
REQ8	3		X			
REQ9	2					X
REQ10	5				X	
REQ11	4		X		X	
REQ12	2	X	X	X	X	X
Max PW		5	5	5	5	5
Total PW		11	14	12	17	11

System Architecture and System Design

Architectural Styles

The SeatSphere system follows a three-tier client server architecture that is combined with a RESTful architectural style. The system is divided into a presentation layer, an application layer, and a data storage layer.

The presentation tier is the frontend interface that the user will interact with through the web browser. This interface will allow customers to view movies, select seats, and purchase tickets. It will also allow administrators to manage the movies and the seating information.

The application tier is the backend server that processes the request from the frontend. The backend is implemented using the Java Spring Boot, which handles the systems business logic such as processing bookings, managing seats, and getting information from the database.

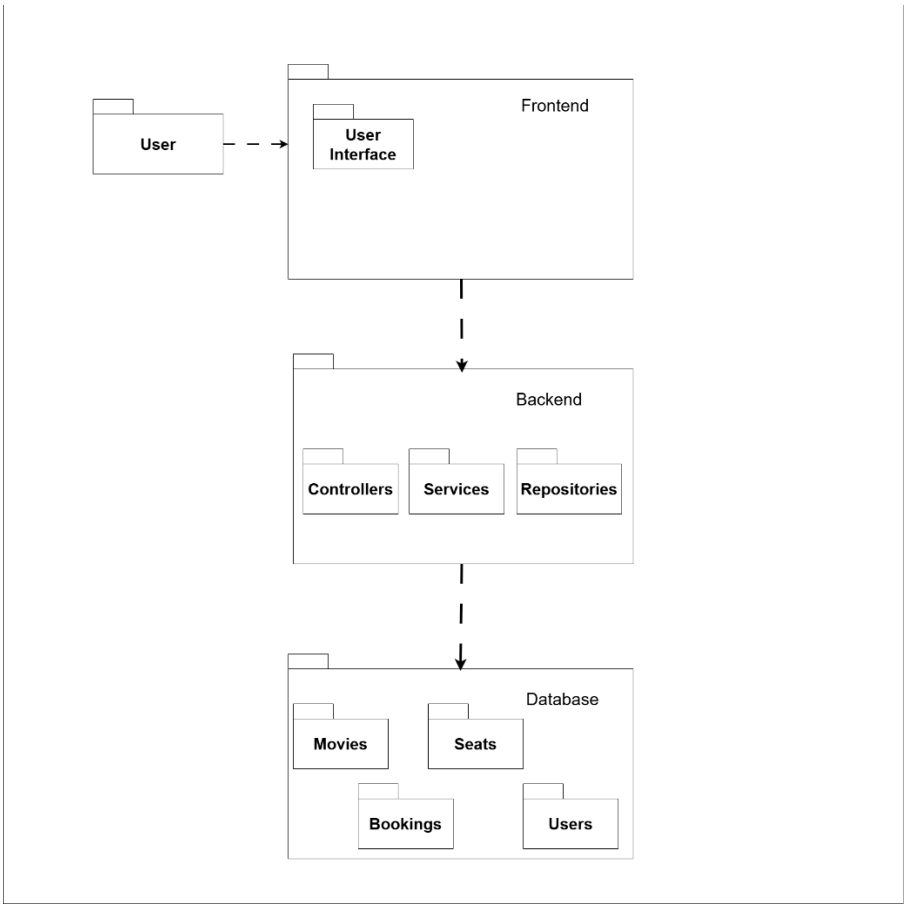
The data tier is a MySQL relational database that stores the system data. This includes information about movies, seats, users, and bookings.

The front end communicates with the backend using a REST API where the request are sent using HTTP and responses are thus returned in JSON format.

Identifying Subsystems

The SeatShere system can be divided into three main subsystems: Frontend, Backend, and Database. The Frontend Subsystem manages the user interface and runs in the user's web browser. It displays movie information which allows the users to choose seats and sends requests to the backend server. The Backend Subsystem process system operations. It contains the controller layers, service layers, and repository layer. These components manage API requests, perform business logic, and interact with the database. The Database Subsystem stores all persistent information for the system. The dataset will keep track of movies, seat availability, users, and ticket bookings. A UML package diagram shown below details the Frontend

Subsystem communicating with the Backend Subsystem through REST API calls, while the Backend Subsystem communicates with the Database Subsystem through database queries.



Persistent Data Storage

The SeatSphere system requires persistent data storage so that the information will remain available after the system stops running. The system uses a relational database management system to store this information. The database is implemented using MySQL, where the dta is organized into tables with relationships between them. The system stores several types of persistent objects, including:

- Movies (title, genre, duration, price, rating, movie poster)
- Seats (if/is occupied, seat number, seat row, if handicap)
- Showtimes (start time)

- Theater Halls (name, total seats)
- Tickets (QR code)
- Users (password, username, role, email, first name, last name)
- Bookings (reservation records)

Network Protocol

The SeatSphere system uses HTTP and HTTPS for communication between the frontend and backend. The HTTP standard protocol that is used by these web applications allow the frontend to send requests over to the backend server. The backend provides a REST API that will handle these requests. The data that is exchanged between the frontend and the backend is formatted using JSON. For example, GET request retrieve information such as movies or seats. The POST request are used to create the bookings or tickets.

Global Control Flow

Execution Orders

The event driven system waits for users actions such as clicking buttons or submitting forms. While these actions send a request to the backend, it process the request and returns a response. Many different users may perform these actions in different orders. For example, one user might browse the movies while another might be selecting seats.

Time Dependency

While there is not a strict real-time requirement, the system instead follows an event response model. This is where the system reacts to users request when they occur and allows the system to track movie showtimes to help determine whether seats can be booked for a specific screening.

Concurrency

Multiple users are supported by SeatSphere while accessing the system at the same time. The backend will process each request using separate threads. This helps allow the users to browse movies and reserve seats simultaneously.

Hardware Requirements

- *Display:* A color display with a minimum resolution of 1024 x 768 pixels for proper rendering of the Admin Dashboard.
- *Memory:* A minimum of 4 GB of RAM to support the Java Virtual Machine, Node.js runtime, and MySQL engine.
- *Hard Drive:* At least 500 MB of available disk space for the JDK, Node modules, and the MySQL data directory.
- *Network*: A minimum network bandwidth of 100 Kbps for reliable API communication between subsystems.
- *Software Runtime*: Java Runtime Environment (JRE) 17+, Node.js 18+, and MySQL 8.0+.

Plan of Work / Development Plan

Week 1 – 2	○ Determine the framework and establish the structure of the system. ○ Set up the Node.js environment and Java Spring Boot skeleton. ○ Establish the connection between the backend and the MySQL database.	✅

Week 3-4	○ Build the system login and registration for both customers and theater managers. ○ Implement secure password hashing and session management in the Java backend.	✅ Verified user registration through Postman API testing
Week 5-7	○ Implement basic features for theater managers (SeatSphere): allow managers to add, edit, and delete movie listings by title, genre, or ID. ○ Provide tools to create and manage showtimes and assign them to specific theater halls.	✅
Week 8	○ Test the main administrative and authentication features accomplished. Record the demo for the mid-term.	✅
Week 9-11	○ Improve current features based on feedback or continue working on basic features for moviegoers (HappySeats): the system will allow users to browse movies by showtime. ○ Users will select seats from a real-time visual map. ○ The system will record the booking in the database, calculate the total price, and provide a digital ticket confirmation.	

Week 12-14	○ Writing test cases for implemented features (specifically testing to ensure two users cannot book the same seat). ○ Finish building any additional features like booking history or "cancel reservation" logic.	
Week 15	○ Record the demo for the final presentation.	

References

Academic & Design Foundations

Tilley, S., & Rosenblatt, H. (2017). *Systems analysis and design* (11th ed.). Cengage.

Wireframe.cc. (n.d.). *Wireframe.cc* [Computer software]. https://wireframe.cc/

Nielsen, J. (1994). *Enhancing the explanatory power of usability heuristics*. Proceedings of the ACM CHI'94 Conference.

Data Management & Backend Infrastructure

Hibernate. (2024). *Hibernate ORM 6.4* [Object-relational mapping software]. Red Hat. https://hibernate.org/

Oracle Corporation. (2023). *Java Platform, Standard Edition 17* [Software development kit]. https://www.oracle.com/java/

Oracle Corporation. (2024). *MySQL Community Server* (Version 8.0) [Database management system]. https://www.mysql.com/

VMware Tanzu. (2024). *Spring Boot* (Version 3.2) [Software framework]. https://spring.io/projects/spring-boot

VMware Tanzu. (2024). *Spring Security* (Version 6.2) [Authentication and access control framework]. https://spring.io/projects/spring-security

Frontend Development & Middleware

OpenJS Foundation. (2024). *Node.js* (Version 20.x) [JavaScript runtime environment]. https://nodejs.org/

StrongLoop. (2024). *Express* (Version 4.x) [Web application framework]. https://expressjs.com/

WHATWG. (n.d.). *Fetch Living Standard* [Technical specification]. https://fetch.spec.whatwg.org/

Quality Assurance & Build Tools

Apache Software Foundation. (2024). *Apache Maven* (Version 3.9) [Build automation tool]. https://maven.apache.org/

JUnit Team. (2024). *JUnit 5* (Version 5.10) [Unit testing framework]. https://junit.org/junit5/

Microsoft. (2026). *Visual Studio Code* [Source-code editor]. https://code.visualstudio.com/